

Table des matières

Introduction générale	1
1 Organisme d'accueil et système d'information	2
Introduction	3
1.1 Présentation de l'organisme d'accueil	3
1.1.1 Présentation de Toulouse INP-ENSEEIH	3
1.1.2 Le service des relations internationales	3
1.2 Analyse du système d'information interne	4
1.2.1 Les outils en place	4
1.2.1.1 QS MoveON	4
1.2.1.2 Pégase	4
1.2.1.3 Eudonet	4
1.2.1.4 Les fichiers Excel	5
1.2.2 Limites structurelles	5
Conclusion	6
2 Analyse des solutions existantes et positionnement	7
Introduction	8
2.1 Grille d'évaluation	8
2.2 Analyse des solutions existantes	8
2.2.1 QS MoveON	9
2.2.2 ErasmusJET	9
2.2.3 Travaux académiques : Elahi et al. (2022)	10
2.3 Tableau comparatif de synthèse	10
2.4 Problématique et solution proposée	11
2.4.1 Problématique	11
2.4.2 Solution proposée	12
Conclusion	12
3 Analyse et spécification du système	13
Introduction	14
3.1 Besoins fonctionnels	14
3.1.1 Fonctionnalités opérationnelles	14
3.1.2 Fonctionnalités intelligentes	14
3.2 Besoins non fonctionnels	15
3.3 Profils utilisateurs et périmètre de réalisation	16

3.3.1	Profils utilisateurs	16
3.3.2	Périmètre de réalisation	16
3.4	Cadre méthodologique et approches de développement	17
3.4.1	Méthodologie de gestion de projet	17
3.4.1.1	Méthode en cascade	17
3.4.1.2	Kanban	17
3.4.1.3	Scrum	17
3.4.1.4	Comparaison et choix retenu	18
3.4.2	Organisation du code : Domain-Driven Design allégé	18
3.4.2.1	Découpage en domaines métier	18
3.4.2.2	Langage commun	19
3.4.2.3	Encapsulation des règles métier	19
3.4.3	Style architectural : architecture en couches	19
	Conclusion	20
4	Choix des technologies	21
	Introduction	23
4.1	Choix du frontend	23
4.1.1	Besoins spécifiques	23
4.1.2	Alternatives évaluées	23
4.1.2.1	Applications monopage — React + Vite	23
4.1.2.2	Angular	24
4.1.2.3	Next.js 14	24
4.1.3	Synthèse comparative	25
4.1.4	Décision	25
4.2	Choix du backend	25
4.2.1	Besoins spécifiques	25
4.2.2	Choix du framework backend	26
4.2.2.1	Alternatives évaluées	26
4.2.2.1.1	Node.js	26
4.2.2.1.2	FastAPI	26
4.2.2.1.3	Django	27
4.2.2.2	Synthèse comparative	27
4.2.2.3	Décision	28
4.2.3	Choix de la couche API	28
4.2.3.1	Besoins spécifiques	28
4.2.3.2	Alternatives évaluées	28
4.2.3.2.1	Django REST Framework (DRF)	28
4.2.3.2.2	Django Ninja	29
4.2.3.3	Synthèse comparative	29

4.2.3.4	Décision	30
4.3	Choix de la base de données	30
4.3.1	Besoins spécifiques	30
4.3.2	Alternatives évaluées	30
4.3.2.1	MySQL	30
4.3.2.2	MongoDB	31
4.3.2.3	PostgreSQL 16	31
4.3.3	Synthèse comparative	32
4.3.4	Décision	32
4.4	Choix du format de stockage des imports bruts	33
4.4.1	Besoins spécifiques	33
4.4.2	Alternatives évaluées	33
4.4.2.1	Fichiers plats (JSON sur le système de fichiers)	33
4.4.2.2	Base NoSQL dédiée (MongoDB)	33
4.4.2.3	Type JSONB dans PostgreSQL	34
4.4.3	Synthèse comparative	34
4.4.4	Décision	34
4.5	Choix du gestionnaire de tâches asynchrones	35
4.5.1	Besoins spécifiques	35
4.5.2	Alternatives évaluées	35
4.5.2.1	Celery + Redis	35
4.5.2.2	Django-Q2 + PostgreSQL	36
4.5.3	Synthèse comparative	36
4.5.4	Décision	37
4.6	Choix du stockage de fichiers	37
4.6.1	Besoins spécifiques	37
4.6.2	Alternatives évaluées	37
4.6.2.1	Système de fichiers local	37
4.6.2.2	MinIO (stockage objet S3-compatible)	37
4.6.3	Synthèse comparative	38
4.6.4	Décision	38
4.7	Couche de routage des requêtes	39
4.7.1	Besoins spécifiques	39
4.7.2	Alternatives évaluées	39
4.7.2.1	Reverse proxy	39
4.7.2.2	API Gateway	39
4.7.3	Synthèse comparative	40
4.7.4	Décision	40
4.7.5	Choix de l'implémentation : Nginx vs Traefik	40
4.7.5.1	Besoins spécifiques	40

4.7.5.2	Alternatives évaluées	40
4.7.5.2.1	Nginx	41
4.7.5.2.2	Traefik	41
4.7.5.3	Synthèse comparative	41
4.7.5.4	Décision	42
4.8	Synthèse des choix technologiques	42
	Conclusion	43
5	Choix algorithmiques et stratégies de performance	44
	Introduction	46
5.1	Stratégie de performance base de données	46
5.1.1	Contexte et enjeux	46
5.1.2	Alternatives évaluées	46
5.1.2.1	Partitionnement par année	46
5.1.2.2	Indexation ciblée	46
5.1.3	Synthèse comparative	47
5.1.4	Décision	47
5.2	Choix de l'algorithme d'affectation	47
5.2.1	Besoins spécifiques	48
5.2.2	Alternatives évaluées	48
5.2.2.1	Algorithme glouton	48
5.2.2.2	Gale-Shapley	48
5.2.3	Synthèse comparative	49
5.2.4	Décision	49
5.3	Choix de la méthode de répartition des quotas par département	50
5.3.1	Besoins spécifiques	50
5.3.2	Alternatives évaluées	50
5.3.2.1	Méthode Hamilton (plus grands restes)	50
5.3.2.2	Méthode Jefferson (plus grandes moyennes)	51
5.3.2.3	Méthode Webster	51
5.3.3	Synthèse comparative	51
5.3.4	Décision	52
5.4	Choix du moteur de recommandation	52
5.4.1	Besoins spécifiques	52
5.4.2	Alternatives évaluées	52
5.4.2.1	Approche basée sur des règles (rule-based)	53
5.4.2.2	Approche par similarité (collaborative filtering)	53
5.4.2.3	Approche par apprentissage supervisé	53
5.4.2.3.1	Régression logistique	53
5.4.2.3.2	XGBoost	54

5.4.3	Synthèse comparative	54
5.4.4	Décision	54
5.5	Choix de l'orchestration RAG	55
5.5.1	Besoins spécifiques	55
5.5.2	Modèle de langage	55
5.5.3	Alternatives évaluées	56
5.5.3.1	LangChain	56
5.5.3.2	LlamaIndex	56
5.5.3.3	Haystack	56
5.5.3.4	Approche directe (sans framework)	57
5.5.4	Synthèse comparative	57
5.5.5	Décision	58
5.6	Synthèse des choix	58
	Conclusion	58
6	Conception du système	59
	Introduction	61
6.1	Product Backlog	61
6.1.1	Priorités MoSCoW	61
6.1.2	Story points	61
6.1.3	Tableau du Product Backlog	62
6.2	Planification des sprints	65
6.2.1	Diagramme de Gantt	66
6.3	Architecture logicielle	69
6.3.1	Vue d'ensemble	69
6.3.2	Couche présentation — Next.js 14	70
6.3.3	Couche applicatif — Django 5 + Django Ninja	70
6.3.4	Couche traitement asynchrone — Django-Q2	70
6.3.5	Couche données — PostgreSQL 16	70
6.3.6	Couche intégration — Adaptateurs ETL	71
6.3.6.1	Stratégie d'upsert	71
6.3.6.2	Clé de jointure : l'INE	72
6.3.6.3	Stratégie de synchronisation	72
6.3.7	Sécurité et contrôle d'accès	72
6.3.7.1	Flux d'authentification OIDC	72
6.3.7.2	Contrôle d'accès (RBAC)	73
6.3.7.3	Gestion du renouvellement des tokens JWT	73
6.3.7.4	Journal d'audit	73
6.3.7.5	Protection des données (RGPD)	74
6.4	Architecture physique et déploiement	74

6.4.1	Vue d'ensemble	74
6.4.2	Containers Docker	75
6.4.3	Flux réseau	76
6.4.4	Environnements	76
6.4.5	CI/CD	76
6.4.6	Stratégie de migration de schéma	77
6.4.7	Stratégie de rollback	77
6.4.8	Politique de sauvegarde	77
6.4.9	Gestion des secrets	78
6.5	Modèle de données	78
6.5.1	Organisation par packages	78
6.5.2	Principes structurants	79
6.5.2.1	L'année universitaire comme axe temporel	79
6.5.2.2	L'INE comme identifiant pivot	79
6.5.2.3	Hierarchie quotas INP / quotas N7	79
6.6	Machines à états finis (FSM)	79
6.6.1	Principe et implémentation	79
6.6.2	FSM de l'année universitaire	80
6.6.3	FSM des entités métier	81
6.7	Flux de données : de la source au workflow	81
6.7.1	Détail de l'étape de nettoyage et validation	82
6.7.1.1	Description du processus de transformation	82
6.7.1.1.1	Standardisation des formats	83
6.7.1.1.2	Normalisation sémantique	83
6.7.1.1.3	Gestion des valeurs manquantes	83
6.7.1.1.4	Détection et gestion des doublons	83
6.7.1.1.5	Validation métier	84
6.7.1.1.6	Traçabilité et gestion des erreurs	84
6.7.1.2	Architecture du pipeline de nettoyage	84
6.7.1.3	Technologies et outils utilisés	85
6.8	Gestion des risques	85
	Conclusion	87
7	Réalisation	88
	Introduction	89
7.1	Sprint 1 — Socle technique et référentiels	89
7.1.1	Objectif du sprint	89
7.1.2	Sprint Backlog — Sprint 1	89
7.1.3	Conception détaillée	91
7.1.3.1	Diagramme de classes	91

7.1.4	Implémentation	92
7.1.5	Tests	92
7.2	Sprint 2 — Données et traçabilité	92
7.2.1	Objectif du sprint	92
7.2.2	Sprint Backlog — Sprint 2	93
7.2.3	Conception détaillée	95
7.2.3.1	Diagramme de classes	95
7.2.4	Implémentation	96
7.2.5	Tests	96
7.3	Sprint 3 — Algorithme d’affectation	96
7.3.1	Objectif du sprint	96
7.3.2	Sprint Backlog — Sprint 3	97
7.3.3	Conception détaillée	99
7.3.3.1	Diagramme de classes	99
7.3.4	Implémentation	99
7.3.5	Tests	99
Conclusion générale		100
Bibliographie		101

Table des figures

1.1	Système d'information existant du service RI	5
6.1	Architecture logicielle	69
6.2	Architecture physique	75
6.3	Machine à états — AcademicYear	80
6.4	Pipeline global de traitement des données	82
6.5	Pipeline détaillé de l'étape de nettoyage des données	85
7.1	Diagramme de classes — Sprint 1	92
7.2	Diagramme de classes — Sprint 2	96

Liste des tableaux

2.1	Grille d'évaluation des solutions existantes	8
2.2	Synthèse comparative des solutions selon la grille d'évaluation	11
3.1	Comparaison des approches de gestion de projet	18
3.2	Découpage en domaines métier du système	19
4.1	Comparaison des frameworks frontend	25
4.2	Comparaison des frameworks backend	28
4.3	Comparaison DRF et Django Ninja	30
4.4	Comparaison des systèmes de gestion de base de données	32
4.5	Comparaison des formats de stockage pour RawImport	34
4.6	Comparaison des gestionnaires de tâches asynchrones	36
4.7	Comparaison des solutions de stockage de fichiers	38
4.8	Comparaison des approches de routage des requêtes	40
4.9	Comparaison des outils de reverse proxy	42
4.10	Synthèse des choix technologiques	42
5.1	Comparaison des stratégies de performance base de données	47
5.2	Comparaison des algorithmes d'affectation	49
5.3	Comparaison des méthodes de répartition des quotas	52
5.4	Comparaison des approches de recommandation	54
5.5	Variables d'entrée du moteur de recommandation	55
5.6	Comparaison des frameworks d'orchestration RAG	57
5.7	Synthèse des choix algorithmiques et de performance	58
6.1	Échelle des story points	61
6.2	Product Backlog	62
6.3	Planification des sprints	66
6.4	Diagramme de Gantt	67
6.5	Stratégie de synchronisation par source	72
6.6	Containers Docker et leurs rôles	76
6.7	Packages du modèle de données	78
6.8	Conditions préalables (guard conditions) des transitions FSM	81
6.9	FSM des entités métier	81
6.10	Matrice des risques	86

Liste des abréviations

CAS	Central Authentication Service
CTI	Commission des Titres d'Ingénieur
GPA	Grade Point Average
INP	Institut National Polytechnique
RI	Relations Internationales

Introduction générale

Organisme d'accueil et système d'information

Plan

Introduction	3
1.1 Présentation de l'organisme d'accueil	3
1.1.1 Présentation de Toulouse INP-ENSEEIH	3
1.1.2 Le service des relations internationales	3
1.2 Analyse du système d'information interne	4
1.2.1 Les outils en place	4
1.2.1.1 QS MoveON	4
1.2.1.2 Pégase	4
1.2.1.3 Eudonet	4
1.2.1.4 Les fichiers Excel	5
1.2.2 Limites structurelles	5
Conclusion	6

Introduction

Ce chapitre présente l’organisme d’accueil, le service des Relations Internationales de l’ENSEEIH, et décrit les outils actuellement utilisés pour la gestion des mobilités, ainsi que les limites structurelles qui ont motivé ce projet.

1.1 Présentation de l’organisme d’accueil

Ce stage a été effectué au sein de Toulouse INP-ENSEEIH, école d’ingénieurs publique toulousaine, et plus précisément dans son service des Relations Internationales. Cette section présente d’abord l’établissement d’accueil, puis le service dans lequel le stage a été réalisé.

1.1.1 Présentation de Toulouse INP-ENSEEIH

Toulouse INP-ENSEEIH est une école publique d’ingénieurs, située à Toulouse et rattachée à l’Institut national polytechnique de Toulouse. Elle forme chaque année des ingénieurs, des étudiants en master et en doctorat dans trois spécialités : Sciences du Numérique (SN), Électronique–Énergie électrique–Automatique (EEEA) et Mécanique des Fluides, Énergétique & Environnement (MFEE).

L’école collabore avec plusieurs laboratoires de recherche, notamment IMFT, IRIT, LAAS et LAPLACE, et entretient des liens avec de nombreux partenaires industriels. Le développement à l’international fait partie de ses priorités : elle agrandit régulièrement son réseau d’universités partenaires et favorise les séjours à l’étranger pour ses étudiants [1].

1.1.2 Le service des relations internationales

Le service des Relations Internationales (RI) gère les échanges entre l’ENSEEIH et ses partenaires à l’étranger. Son rôle est à la fois stratégique et opérationnel : il s’occupe des accords de partenariat, mais aussi du suivi quotidien des mobilités étudiantes. Ses activités se répartissent en six domaines [2].

La gestion des partenariats académiques : gestion d’un réseau d’accords bilatéraux avec des universités étrangères, incluant les programmes d’échange et les doubles diplômes, avec suivi des places disponibles par accord et par département.

L’organisation des mobilités sortantes : organisation de la campagne annuelle de sélection des étudiants de l’ENSEEIH partant à l’étranger, depuis la publication des destinations jusqu’à la validation des affectations [3].

L’accueil des mobilités entrantes : inscription et suivi des étudiants étrangers accueillis à l’ENSEEIH dans le cadre des accords de partenariat [4].

L’accompagnement administratif et pédagogique : aide aux étudiants pour les procédures de départ, suivi des aspects financiers et vérification de la conformité des dossiers.

Le reporting CTI : suivi des mobilités entrantes et sortantes pour les audits de la Commission des Titres d’Ingénieur, qui exige une durée minimale de séjour à l’étranger pour l’obtention du diplôme.

La gestion des programmes courts : suivi des étudiants participant à des summer schools, séjours linguistiques ou autres programmes de courte durée, qui peuvent être comptabilisés dans la durée de mobilité CTI sous certaines conditions [5].

Ces activités nécessitent de traiter chaque jour des données venant de plusieurs sources différentes, dans des formats qui ne sont pas toujours compatibles. C’est ce problème qui est au cœur du projet présenté dans ce rapport.

1.2 Analyse du système d’information interne

Avant de proposer une solution, il faut d’abord comprendre comment le service RI fonctionne et quels outils il utilise. Cette section décrit les systèmes en place, puis présente les problèmes que leur utilisation crée au quotidien.

1.2.1 Les outils en place

Le service RI utilise quatre outils distincts, qui ne communiquent pas entre eux.

1.2.1.1 QS MoveON

MoveON est l’outil principal du service RI. Il permet de gérer les accords bilatéraux, les quotas INP et les candidatures des étudiants. Il dispose d’une API REST qui permet d’extraire les données [6]. Mais il ne prend pas en compte les résultats académiques des étudiants, ne propose pas d’aide pour la sélection, et ne couvre ni les stages ni les autres éléments du parcours individuel.

1.2.1.2 Pégase

Pégase est le système d’information académique de Toulouse INP. Il regroupe les données des étudiants : inscription, département, niveau, parcours pédagogique et notes — en particulier le GPA trimestriel, qui sert à classer les candidatures. Ces informations sont essentielles pour la gestion des mobilités. Les données sont accessibles via une API, mais il n’existe aucun lien direct avec MoveON.

1.2.1.3 Eudonet

Eudonet est l’outil utilisé pour gérer les conventions de stage. Il contient les informations sur les stages nationaux et internationaux : entreprise d’accueil, dates et durée [7]. Ces données entrent dans le calcul de la durée de mobilité CTI, mais elles ne sont pas reliées aux autres systèmes.

1.2.1.4 Les fichiers Excel

Pour pallier les manques des trois outils précédents, le service RI utilise des fichiers Excel qu'il gère lui-même : suivi des quotas par département, gestion des mobilités entrantes, calcul des durées et suivi des programmes courts. Même avec une personne rigoureuse, cette méthode reste risquée : erreurs de saisie, oublis de mise à jour, fautes de frappe, incohérences entre fichiers — ces problèmes sont difficiles à repérer et peuvent avoir des conséquences directes sur le déroulement de la campagne.

1.2.2 Limites structurelles

La figure 1.1 montre l'organisation actuelle du service RI : quatre outils utilisés séparément, sans aucune connexion automatique entre eux. Chaque transfert de données se fait manuellement.

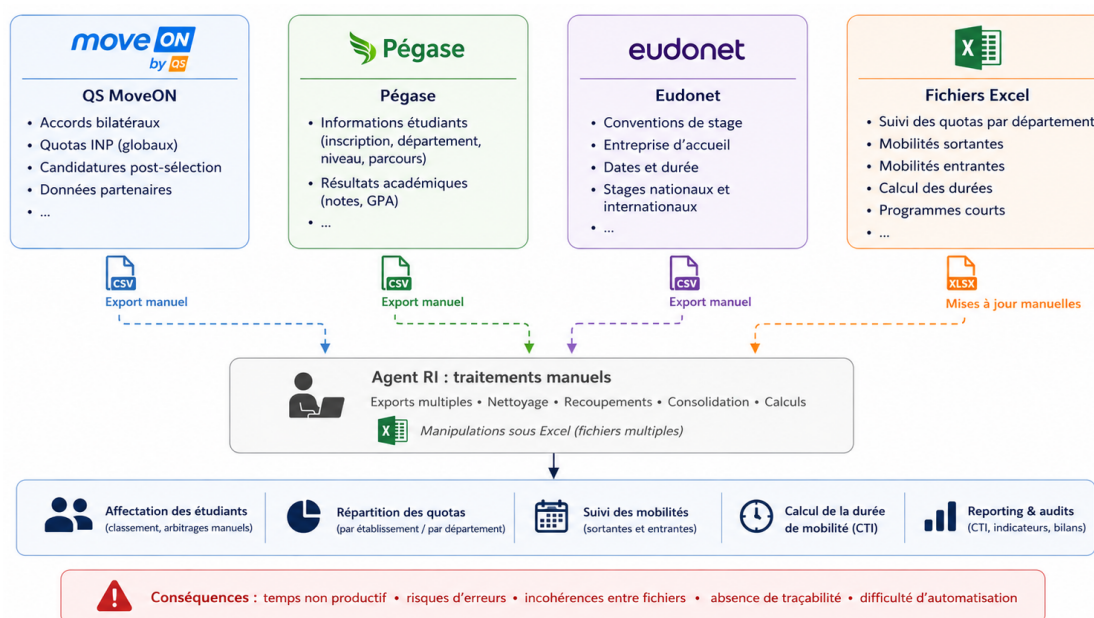


FIGURE 1.1 – Système d'information existant du service RI

Cette absence de communication entre les outils crée plusieurs problèmes qui reviennent à chaque campagne.

- **Des exports manuels à chaque étape.** Comme les outils ne communiquent pas entre eux, chaque consolidation de données nécessite des exports depuis plusieurs plateformes, puis une mise en forme dans Excel. Ce travail est long et source d'erreurs.
- **Une sélection sans règles fixes.** Les candidats sont classés manuellement à partir d'un tableau de GPA. En cas d'égalité ou de cas particulier, c'est l'agent RI qui tranche, et sa décision peut changer d'une année à l'autre sans qu'il y ait de trace écrite.
- **Des quotas insuffisamment détaillés.** MoveON ne gère les places qu'à l'échelle de l'INP global, sans distinguer les établissements qui le composent. La répartition

par établissement (dont l'ENSEEIHT), puis par département pédagogique, est faite manuellement dans Excel, sans vérification de cohérence.

- **Des mobilités complémentaires non suivies.** Les summer schools, séjours linguistiques et programmes courts n'ont pas de place dans les outils existants. Pour les inclure dans le calcul de la durée CTI, l'agent doit s'en souvenir lui-même — ce qui n'est pas fiable.
- **Un calcul de durée CTI entièrement manuel.** Pour calculer la durée de mobilité d'un étudiant, il faut rassembler des données réparties dans trois systèmes différents. Cette opération prend du temps, et des périodes peuvent être oubliées. Elle ne peut pas être automatisée tant qu'il n'existe pas de source de données unifiée.

Conclusion

Les outils utilisés par le service RI fonctionnent chacun de leur côté, sans échange automatique de données. Cela oblige à des traitements manuels sous Excel, sources d'erreurs et difficiles à maintenir. C'est cette situation qui a motivé le développement de la solution présentée dans ce rapport.

Analyse des solutions existantes et positionnement

Plan

Introduction	8
2.1 Grille d'évaluation	8
2.2 Analyse des solutions existantes	8
2.2.1 QS MoveON	9
2.2.2 ErasmusJET	9
2.2.3 Travaux académiques : Elahi et al. (2022)	10
2.3 Tableau comparatif de synthèse	10
2.4 Problématique et solution proposée	11
2.4.1 Problématique	11
2.4.2 Solution proposée	12
Conclusion	12

Introduction

Ce chapitre présente les solutions existantes afin d'évaluer leur capacité à répondre aux besoins identifiés. Il permet de déterminer leur pertinence dans le contexte du service RI avant d'envisager une nouvelle solution.

2.1 Grille d'évaluation

Pour comparer les solutions de façon rigoureuse, une grille de huit critères a été définie à partir des problèmes concrets identifiés au chapitre précédent. Chaque critère traduit une lacune précise du système actuel.

#	Critère	Ce que cela signifie concrètement pour l'ENSEEIH
C1	Centralisation des données de mobilité	Regrouper dans une seule plateforme l'ensemble des données relatives aux mobilités sortantes, entrantes, complémentaires et aux stages internationaux, ainsi que les données académiques des étudiants
C2	Quotas par école et par département	Gestion des places allouées d'abord à l'échelle de l'ENSEEIH, puis par département pédagogique (SN, 3EA, MF2E), et non uniquement à l'échelle globale INP
C3	Aide à la décision et auto-affectation	Présence d'un mécanisme automatique de classement et de pré-affectation des candidats.
C4	Calcul de la durée de mobilité	Agrégation automatique de toutes les périodes de mobilité d'un étudiant (académique, stage, complémentaire) pour produire la durée réglementaire
C5	Mobilités complémentaires	Prise en charge des expériences hors accord bilatéral (summer schools, séjours linguistiques, programmes courts) avec dépôt et validation de justificatifs
C6	Adaptabilité aux règles internes	Capacité du système à fonctionner selon les règles propres à l'établissement : priorités d'affectation, seuils de sélection, critères locaux
C7	Recommandation de destinations	Suggestion de destinations adaptées au profil de l'étudiant (département, GPA, langue, préférences) pour l'accompagner dans son choix
C8	Reporting CTI	Production automatique des indicateurs clés (KPIs) exigés par la CTI : taux de mobilité par promotion, durées moyennes, répartition géographique

TABLE 2.1 – Grille d'évaluation des solutions existantes

2.2 Analyse des solutions existantes

Trois solutions ont été examinées : QS MoveON, déjà utilisé par le service RI, Erasmus-JET, développé dans le cadre du programme Erasmus+, et les travaux académiques d'Elahi et al. sur la recommandation de destinations.

2.2.1 QS MoveON

MoveON est l'outil central du service RI pour la gestion des accords bilatéraux et des candidatures. Il est examiné ici non pas pour le remplacer, mais pour identifier précisément ce qu'il ne couvre pas [6].

Points forts : le référentiel des accords bilatéraux, le suivi des candidatures post-sélection et l'administration des dossiers étudiants. Son API REST est un point d'appui utile pour extraire les données.

Limites par rapport aux critères définis :

- MoveON se limite aux mobilités sortantes et aux accords. Les données académiques (GPA, département, parcours), les stages internationaux et les mobilités complémentaires en sont totalement absents. Le service RI est donc contraint d'exporter depuis MoveON, de compléter à la main dans Excel, puis de travailler sur ce fichier hybride — ce cumul de gestes est exactement ce que ce projet cherche à éliminer.
- Les quotas ne sont gérés qu'à l'échelle INP. La déclinaison par école (ENSEEIHT) puis par département pédagogique, qui conditionne l'affectation interne, est entièrement tenue dans des fichiers Excel sans aucune contrainte de cohérence.
- Aucun classement automatique n'existe. La sélection est intégralement manuelle, ce qui signifie que les règles appliquées varient selon l'agent et la campagne.
- Sans les stages ni les expériences complémentaires, le calcul de la durée de mobilité est impossible à automatiser depuis MoveON seul.
- Aucune suggestion de destinations adaptée au profil de l'étudiant.
- Les exports MoveON ne produisent pas les indicateurs CTI structurés (taux de mobilité par promotion, durées moyennes, répartition géographique) dans un format directement exploitable pour les audits.

MoveON reste un bon outil pour ce qu'il fait, mais sa conception ne prévoit pas de couvrir les besoins opérationnels internes d'un établissement.

2.2.2 ErasmusJET

ErasmusJET est une plateforme développée dans le cadre des initiatives de digitalisation du programme Erasmus+. Son objectif est de standardiser les échanges de données entre établissements partenaires du réseau européen.

Points forts : la gestion complète du cycle Erasmus — dépôt de candidatures en ligne, génération et signature des conventions, échange de relevés de notes entre institutions. Pour un établissement dont toutes les mobilités relèvent du programme Erasmus, c'est une solution adaptée.

Limites par rapport aux critères définis :

- ErasmusJET ne fonctionne que pour Erasmus+. Les accords hors Europe — qui représentent une grande partie des mobilités de l’ENSEEIH — restent en dehors de son périmètre, tout comme les données académiques, les stages et les mobilités complémentaires.
- Les quotas par département, l’auto-affectation et le calcul de durée CTI ne sont pas prévus dans la plateforme.
- Son format est standardisé pour les échanges européens, ce qui ne laisse pas de place pour les règles propres à chaque établissement.
- Ni recommandation de destinations, ni reporting CTI.

ErasmusJET peut être utile pour la partie Erasmus+, mais il ne règle aucun des problèmes identifiés.

2.2.3 Travaux académiques : Elahi et al. (2022)

Les travaux d’Elahi et al. proposent un système qui combine recommandation de destinations et processus d’affectation étudiante. Les tests sont réalisés sur un jeu de données simulé [8].

Points forts : c’est la seule des trois solutions qui aborde sérieusement le critère C7 — la recommandation personnalisée de destinations. L’article démontre la faisabilité technique de coupler un algorithme de suggestion avec un processus de sélection, ce qui en fait une référence directement pertinente pour la partie intelligence du projet.

Limites par rapport aux critères définis :

- Le modèle repose sur un dataset simulé et uniforme, sans connexion avec des systèmes institutionnels réels. Il ne propose aucune intégration avec MoveON, Pégase ou Eudonet.
- La gestion des quotas par département, l’affectation sous contraintes, le calcul de durée de mobilité et les expériences complémentaires sont entièrement hors du périmètre — le modèle ne traite que la recommandation.
- C’est un prototype de recherche : il n’y a aucun moyen de configurer des règles propres à un établissement.
- Le reporting CTI n’est pas abordé.

Ces travaux sont une référence utile pour la partie recommandation, mais ils ne forment pas une solution prête à être utilisée à l’ENSEEIH.

2.3 Tableau comparatif de synthèse

Le tableau 2.2 résume la couverture de chaque solution pour les huit critères de la grille d’évaluation.

Critère	QS MoveON	ErasmusJET	Elahi et al.
C1 – Centralisation des données	Mobilités sortantes uniquement	✗	✗
C2 – Quotas par école et département	✗	✗	✗
C3 – Aide à la décision et auto-affectation	✗	✗	✗
C4 – Calcul durée de mobilité	✗	✗	✗
C5 – Mobilités complémentaires	✗	✗	✗
C6 – Adaptabilité métier	Règles non adaptables à l'établissement	✗	✗
C7 – Recommandation de destinations	✗	✗	Prototype sur données simulées
C8 – Reporting CTI	✗	✗	✗

TABLE 2.2 – Synthèse comparative des solutions selon la grille d'évaluation

Aucune des solutions étudiées ne répond à l'ensemble des besoins identifiés. Ce résultat confirme la nécessité de développer une solution propre au service RI.

2.4 Problématique et solution proposée

Cette section formule la problématique centrale du projet, puis présente la solution retenue.

2.4.1 Problématique

Le service RI utilise plusieurs outils qui ne communiquent pas entre eux. Chaque transfert de données se fait à la main, l'affectation des étudiants n'a pas de règle fixe, et il n'existe pas de moyen simple de calculer la durée de mobilité. Aucune des solutions analysées dans ce chapitre ne répond à ces besoins. Cinq problèmes restent sans solution :

- Les données sont dispersées entre plusieurs systèmes et ne peuvent pas être consultées depuis un seul endroit.
- Les quotas ne descendent pas jusqu'au niveau département, alors que c'est ce niveau qui détermine qui peut partir et où.
- La sélection des candidats se fait à la main, sans règle définie à l'avance.
- Le calcul de la durée de mobilité oblige à rassembler des données venant de plusieurs sources différentes.
- Les étudiants n'ont pas d'aide pour choisir leurs destinations.

C'est à partir de ce constat que se formule la question centrale du projet :

Comment concevoir une plateforme intégrée qui automatise le processus de gestion des mobilités internationales à l'ENSEEIH, depuis la centralisation des données multi-sources jusqu'à l'affectation algorithmique, tout en garantissant la conformité CTI et l'aide à la décision des étudiants ?

2.4.2 Solution proposée

Ce constat conduit au développement d'une plateforme propre au service RI. Elle ne remplace pas les outils existants, mais se connecte à eux pour récupérer les données automatiquement. Elle permet de gérer les quotas par département, de classer et affecter les étudiants selon des règles définies à l'avance, de calculer la durée de mobilité de façon automatique, et d'aider les étudiants à choisir leurs destinations. L'objectif est de centraliser les données et de réduire le travail manuel.

Conclusion

L'étude des solutions existantes montre qu'aucun des outils analysés ne répond pleinement aux besoins du service RI. Ce qui justifie le développement d'une solution sur mesure, dont les spécifications précises font l'objet du chapitre suivant.

Analyse et spécification du système

Plan

Introduction	14
3.1 Besoins fonctionnels	14
3.1.1 Fonctionnalités opérationnelles	14
3.1.2 Fonctionnalités intelligentes	14
3.2 Besoins non fonctionnels	15
3.3 Profils utilisateurs et périmètre de réalisation	16
3.3.1 Profils utilisateurs	16
3.3.2 Périmètre de réalisation	16
3.4 Cadre méthodologique et approches de développement	17
3.4.1 Méthodologie de gestion de projet	17
3.4.1.1 Méthode en cascade	17
3.4.1.2 Kanban	17
3.4.1.3 Scrum	17
3.4.1.4 Comparaison et choix retenu	18
3.4.2 Organisation du code : Domain-Driven Design allégé	18
3.4.2.1 Découpage en domaines métier	18
3.4.2.2 Langage commun	19
3.4.2.3 Encapsulation des règles métier	19
3.4.3 Style architectural : architecture en couches	19
Conclusion	20

Introduction

Ce chapitre analyse les besoins fonctionnels et non fonctionnels du système, identifie les utilisateurs concernés et présente les choix méthodologiques qui ont guidé le développement.

3.1 Besoins fonctionnels

Les besoins fonctionnels décrivent les principales fonctionnalités attendues du système afin de répondre aux problématiques identifiées au sein du service RI. Ils couvrent à la fois les fonctionnalités opérationnelles et les fonctionnalités intelligentes

3.1.1 Fonctionnalités opérationnelles

Le système prend en charge les opérations suivantes, qui couvrent l'ensemble du cycle de gestion des mobilités internationales, de l'import des données jusqu'à l'archivage des résultats.

- **Intégration des données** : import depuis MoveON, Pégase et Eudonet via API REST, et depuis des fichiers Excel pour les sources sans API. Chaque import inclut une détection de doublons, une validation des formats et un mécanisme de fallback Excel en cas d'indisponibilité d'une API.
- **Gestion des accords et quotas** : import des accords depuis MoveON et gestion de la répartition locale des places par département N7, avec possibilité d'ajustement manuel par le service RI.
- **Algorithme d'affectation** : pré-affectation automatique basée sur le GPA, les vœux ordonnés et les quotas par département, avec interface de supervision et d'ajustement.
- **Suivi CTI** : calcul automatique de la durée totale de mobilité par agrégation des séjours académiques, stages et expériences complémentaires validées.
- **Mobilités complémentaires** : les étudiants déclarent leurs expériences et déposent les justificatifs nécessaires, puis le service RI se charge de la validation des dossiers.
- **Mobilités entrantes** : import via template Excel et suivi administratif granulaire.
- **Historisation** : les données d'une campagne clôturée passent en lecture seule. Aucune exécution ultérieure ne peut modifier les résultats d'une année passée.

3.1.2 Fonctionnalités intelligentes

En complément des fonctionnalités opérationnelles, le système intègre deux modules intelligents qui améliorent l'expérience étudiant sans solliciter le service RI.

- **Moteur de recommandation** : suggestions de destinations basées sur le GPA, le département et l'historique des affectations, pour orienter les choix de l'étudiant avant la saisie dans MoveON.
- **Assistant virtuel (chatbot RAG)** : agent conversationnel répondant aux questions procédurales à partir de la documentation institutionnelle, sans solliciter le service RI.

3.2 Besoins non fonctionnels

Au-delà des fonctionnalités métier, le système doit également répondre à plusieurs exigences techniques et organisationnelles. Ces besoins non fonctionnels concernent notamment la sécurité, la performance, l'intégrité des données et l'intégration avec l'environnement existant.

- **Interopérabilité** : le système doit s'intégrer sans rupture dans l'écosystème existant via des adaptateurs REST dédiés à chaque source. En cas d'indisponibilité d'une API, un fallback par import Excel garantit la continuité opérationnelle.
- **Intégrité des données** : les données provenant de sources hétérogènes peuvent être incohérentes ou dupliquées. Une validation stricte est appliquée à chaque import — contrôle de format, détection de doublons, cohérence inter-sources. Toute donnée rejetée est tracée dans un rapport d'import avec la raison précise, pour correction ciblée.
- **Sécurité et RGPD** : authentification via le SSO institutionnel, gestion stricte des rôles (étudiant vs administrateur), chiffrement des données sensibles. Un étudiant n'accède qu'à son propre dossier.
- **Performance** : le temps de réponse de l'API pour les endpoints courants doit être inférieur à 200 ms sous une charge de 50 utilisateurs simultanés. L'algorithme d'affectation doit s'exécuter en moins de 30 secondes pour une promotion de 300 étudiants. Les traitements lourds — algorithme d'affectation, calculs CTI — sont exécutés de façon asynchrone en arrière-plan pour ne pas bloquer l'interface.
- **Scalabilité** : séparation entre données actives et données archivées en lecture seule pour absorber la croissance annuelle du volume historique et les pics de connexion lors des moments clés de campagne.
- **Ergonomie** : l'interface adapte sa navigation et ses fonctionnalités au rôle de l'utilisateur connecté, réduisant les erreurs de manipulation.
- **Traçabilité** : toute action importante — import, modification d'affectation, validation, transition de phase — est enregistrée avec l'identité de l'auteur et l'horodatage dans un journal d'audit persistant en écriture seule.

3.3 Profils utilisateurs et périmètre de réalisation

Cette section présente les profils d'utilisateurs du système, puis définit le périmètre de réalisation retenu pour ce projet.

3.3.1 Profils utilisateurs

Deux profils utilisent le système, avec des besoins très différents.

L'administrateur — c'est-à-dire le service RI — pilote la campagne de bout en bout. Il supervise les imports, configure les quotas, ajuste les propositions d'affectation, instruit les demandes de mobilités complémentaires et produit les rapports CTI. Le système prend en charge les tâches répétitives, mais toute décision finale reste sous sa responsabilité, et chaque intervention manuelle est tracée.

L'étudiant consulte les destinations disponibles, reçoit des recommandations personnalisées avant de saisir ses vœux dans MoveON, suit l'état de sa candidature, déclare ses expériences complémentaires et vérifie à tout moment sa conformité CTI. L'assistant virtuel répond à ses questions procédurales sans mobiliser le service RI.

3.3.2 Périmètre de réalisation

Le développement est organisé en deux niveaux de priorité.

MVP — fonctionnalités prioritaires :

- Intégration des données multi-sources (MoveON, Pégase, Eudonet, Excel)
- Gestion des accords et quotas par département
- Algorithme d'affectation avec validation humaine
- Calcul automatique de la durée de mobilité CTI
- Gestion des mobilités complémentaires et entrantes
- Tableaux de bord analytiques avancés et exports CTI
- Espace étudiant : catalogue, suivi de candidature, déclarations
- Sécurité : SSO, contrôle d'accès, audit log

V2 — fonctionnalités secondaires :

- Moteur de recommandation de destinations personnalisé
- Assistant virtuel (chatbot RAG)

3.4 Cadre méthodologique et approches de développement

La conduite du projet repose sur trois niveaux complémentaires : une méthodologie de gestion de projet, une approche d'organisation du code, et un style architectural.

3.4.1 Méthodologie de gestion de projet

Le projet s'étend sur six mois dans un contexte où certaines règles métier n'étaient pas complètement formalisées dès le départ. Trois approches ont été évaluées avant de retenir la méthodologie la mieux adaptée.

3.4.1.1 Méthode en cascade

La méthode en cascade organise le projet en phases séquentielles et strictement ordonnées : analyse, conception, développement, test, déploiement. Chaque phase doit être complètement terminée avant de passer à la suivante.

Avantages : Elle offre une planification claire et une documentation exhaustive dès le départ, ce qui facilite le suivi et la traçabilité des décisions.

Limites dans notre contexte : Elle impose de geler un cahier des charges complet avant tout développement. Or, certaines règles métier — notamment la hiérarchie des quotas et les critères de priorité dans l'affectation — n'étaient pas entièrement définies au démarrage du projet, rendant cette approche inadaptée.

3.4.1.2 Kanban

Kanban est une méthode de gestion de flux continu, sans itérations fixes. Le travail est visualisé sur un tableau et les tâches avancent au fur et à mesure de la disponibilité de l'équipe.

Avantages : Sa flexibilité permet de s'adapter rapidement aux changements et de traiter les tâches par ordre de priorité sans contrainte de cycle.

Limites dans notre contexte : L'absence d'itérations fixes et de jalons structurés le rend peu adapté à un projet avec un périmètre défini et une date de livraison fixe, où une planification rigoureuse est nécessaire.

3.4.1.3 Scrum

Scrum est un cadre agile qui organise le développement en sprints de durée fixe, avec des événements régulières (planning, review, retrospective) et un backlog priorisé.

Avantages : Il offre un bon équilibre entre structure et flexibilité : les priorités peuvent être révisées à chaque sprint, les livraisons sont incrémentales et validables régulièrement.

par le service RI.

Limites dans notre contexte : Scrum est conçu pour des équipes, ce qui nécessite une adaptation des rôles et des événements pour un projet porté par un seul développeur

3.4.1.4 Comparaison et choix retenu

Le tableau 3.1 compare les trois approches et justifie le choix retenu.

Critère	Cascade	Kanban	Scrum
Planification	Rigide dès le début	Continue, sans itérations fixes	Sprints fixes avec rétroplanification
Adaptation aux changements	Difficile	Bonne, mais sans rythme structuré	Bonne, intégrée à chaque sprint
Visibilité client	Faible en cours de projet	Modérée	Élevée (Sprint Review)
Adapté à ce projet	✗	Sans cadence itérative fixe	✓

TABLE 3.1 – Comparaison des approches de gestion de projet

Scrum est retenu pour son équilibre entre structure (sprints de durée fixe, backlog priorisé) et flexibilité (révision des priorités à chaque sprint, livraisons incrémentales validables par le service RI).

Chaque sprint suit un cycle en quatre étapes : un *Sprint Planning* pour sélectionner et décomposer les user stories, un *Daily Standup* adapté au contexte d'un développeur unique sous forme d'un point quotidien d'avancement et de blocages, une *Sprint Review* pour démontrer les fonctionnalités livrées au service RI, et une *Sprint Retrospective* pour ajuster le processus.

Les rôles sont adaptés au contexte académique : le service RI joue le rôle de *Product Owner* (priorisation du backlog, validation des livraisons), l'encadrante académique assure le rôle de *Scrum Master* (suivi de l'avancement, levée des blocages, orientation technique), et l'étudiante prend en charge l'intégralité du développement. Le suivi du backlog est géré avec GitHub Projects, les user stories correspondant à des issues liées aux pull requests.

3.4.2 Organisation du code : Domain-Driven Design allégé

Le système est structuré selon les principes du *Domain-Driven Design* (DDD), une approche qui consiste à organiser le code autour des domaines métier plutôt qu'autour de considérations techniques.

3.4.2.1 Découpage en domaines métier

Le système est décomposé en domaines indépendants, chacun correspondant à une fonction distincte de la gestion de la mobilité. Ce découpage garantit qu'une modification dans

un domaine n’affecte pas les autres.

Domaine	Périmètre
mobility	Conventions de partenariat et quotas
students	Données étudiants et inscriptions
institutions	Universités partenaires
academic	Années académiques et cycle de campagne
reference	Données de référence (pays, départements, niveaux)
incoming	Mobilités entrantes
outgoing	Mobilités sortantes
internships	Stages à l’international
complementary	Formations complémentaires
cti	Données liées aux critères CTI
audit	Traçabilité des actions

TABLE 3.2 – Découpage en domaines métier du système

3.4.2.2 Langage commun

Le DDD préconise l’adoption d’un vocabulaire métier commun, partagé entre la développeuse et l’experte du domaine. Les noms des classes et des méthodes reprennent directement le vocabulaire du service RI ce qui facilite les échanges et réduit les risques d’ambiguïté lors de la validation des fonctionnalités.

3.4.2.3 Encapsulation des règles métier

Chaque domaine encapsule ses propres règles métier, ce qui permet de les faire évoluer indépendamment du reste du système. Cette séparation rend le système plus robuste face aux évolutions de besoins, inévitables dans un contexte de développement itératif où certaines règles — comme les critères de priorité dans l’affectation — se précisent au fil des sprints.

3.4.3 Style architectural : architecture en couches

Le système est structuré selon une architecture en couches, où chaque niveau a une responsabilité clairement délimitée. Ce style architectural a été préféré à une architecture microservices, qui aurait été surdimensionnée pour l’échelle du projet et les contraintes d’hébergement de Toulouse INP.

Le système s’articule autour de cinq couches distinctes :

- **La couche présentation** : responsable du rendu des interfaces et de la gestion des interactions utilisateur.
- **La couche applicative** : porte la logique métier, la validation et l’orchestration des domaines.

- **La couche asynchrone** : prend en charge les traitements longs découplés des requêtes, tels que l'algorithme d'affectation ou les synchronisations nocturnes.
- **La couche données** : assure la persistance relationnelle et le stockage des fichiers justificatifs.
- **La couche intégration** : gère la connexion avec les systèmes externes (MoveON, Pégase, Eudonet) via des adaptateurs dédiés.

Conclusion

Ce chapitre a permis de formaliser les besoins fonctionnels et non fonctionnels du système, d'identifier les profils utilisateurs et de justifier les choix méthodologiques retenus. Ces spécifications constituent le socle sur lequel repose la suite du travail. Le chapitre suivant présente les technologies sélectionnées pour répondre à ces exigences.

Choix des technologies

Plan

Introduction	23
4.1 Choix du frontend	23
4.1.1 Besoins spécifiques	23
4.1.2 Alternatives évaluées	23
4.1.2.1 Applications monopage — React + Vite	23
4.1.2.2 Angular	24
4.1.2.3 Next.js 14	24
4.1.3 Synthèse comparative	25
4.1.4 Décision	25
4.2 Choix du backend	25
4.2.1 Besoins spécifiques	25
4.2.2 Choix du framework backend	26
4.2.2.1 Alternatives évaluées	26
4.2.2.1.1 Node.js	26
4.2.2.1.2 FastAPI	26
4.2.2.1.3 Django	27
4.2.2.2 Synthèse comparative	27
4.2.2.3 Décision	28
4.2.3 Choix de la couche API	28
4.2.3.1 Besoins spécifiques	28
4.2.3.2 Alternatives évaluées	28
4.2.3.2.1 Django REST Framework (DRF)	28
4.2.3.2.2 Django Ninja	29
4.2.3.3 Synthèse comparative	29
4.2.3.4 Décision	30
4.3 Choix de la base de données	30
4.3.1 Besoins spécifiques	30
4.3.2 Alternatives évaluées	30
4.3.2.1 MySQL	30
4.3.2.2 MongoDB	31
4.3.2.3 PostgreSQL 16	31
4.3.3 Synthèse comparative	32
4.3.4 Décision	32

4.4	Choix du format de stockage des imports bruts	33
4.4.1	Besoins spécifiques	33
4.4.2	Alternatives évaluées	33
4.4.2.1	Fichiers plats (JSON sur le système de fichiers)	33
4.4.2.2	Base NoSQL dédiée (MongoDB)	33
4.4.2.3	Type JSONB dans PostgreSQL	34
4.4.3	Synthèse comparative	34
4.4.4	Décision	34
4.5	Choix du gestionnaire de tâches asynchrones	35
4.5.1	Besoins spécifiques	35
4.5.2	Alternatives évaluées	35
4.5.2.1	Celery + Redis	35
4.5.2.2	Django-Q2 + PostgreSQL	36
4.5.3	Synthèse comparative	36
4.5.4	Décision	37
4.6	Choix du stockage de fichiers	37
4.6.1	Besoins spécifiques	37
4.6.2	Alternatives évaluées	37
4.6.2.1	Système de fichiers local	37
4.6.2.2	MinIO (stockage objet S3-compatible)	37
4.6.3	Synthèse comparative	38
4.6.4	Décision	38
4.7	Couche de routage des requêtes	39
4.7.1	Besoins spécifiques	39
4.7.2	Alternatives évaluées	39
4.7.2.1	Reverse proxy	39
4.7.2.2	API Gateway	39
4.7.3	Synthèse comparative	40
4.7.4	Décision	40
4.7.5	Choix de l'implémentation : Nginx vs Traefik	40
4.7.5.1	Besoins spécifiques	40
4.7.5.2	Alternatives évaluées	40
4.7.5.2.1	Nginx	41
4.7.5.2.2	Traefik	41
4.7.5.3	Synthèse comparative	41
4.7.5.4	Décision	42
4.8	Synthèse des choix technologiques	42
	Conclusion	43

Introduction

Ce chapitre présente les technologies choisies pour développer la plateforme. Pour chaque composant, les alternatives disponibles sont comparées sur la base de critères précis, puis le choix retenu est justifié.

4.1 Choix du frontend

Cette section présente les besoins spécifiques du frontend, évalue les alternatives disponibles, puis justifie le choix retenu.

4.1.1 Besoins spécifiques

L’interface du projet doit couvrir deux espaces bien distincts : d’un côté, un espace dédié au service RI pour piloter les campagnes, de l’autre, un espace étudiant pour consulter les destinations, suivre sa candidature et interagir avec le chatbot. Certaines pages affichent des données sensibles (résultats d’affectation, GPA) et doivent être construites côté serveur pour éviter qu’elles ne soient accessibles avant que le rôle de l’utilisateur soit vérifié. D’autres, comme le suivi de candidature ou le chatbot, nécessitent une navigation fluide et réactive. Gérer ces deux types de pages dans le même projet, avec une séparation fiable des accès selon le rôle, est le principal critère de choix.

4.1.2 Alternatives évaluées

Trois frameworks ont été évalués au regard des besoins identifiés.

4.1.2.1 Applications monopage — React + Vite

React permet de construire des interfaces entièrement rendues dans le navigateur [9].

Avantages :

- Simple à configurer et à démarrer
- Très bien documenté, avec de nombreuses ressources disponibles en ligne

Limites :

- Tout est chargé et affiché dans le navigateur : les données sensibles comme les affectations ou les GPA y arrivent avant même que le rôle de l’utilisateur soit contrôlé, ce qui fragilise la séparation entre l’espace étudiant et l’espace RI
- La protection des pages selon le rôle doit être codée manuellement, sans mécanisme intégré pour l’imposer au niveau du routing
- Les appels à l’API Django passent tous par le navigateur, ce qui expose les tokens d’authentification au client

4.1.2.2 Angular

Angular est un framework TypeScript développé par Google, utilisé principalement pour des applications d'entreprise avec beaucoup de logique interne [10].

Avantages :

- Architecture très structurée, adaptée aux applications avec de nombreux écrans et règles métier
- Typage TypeScript natif, ce qui réduit les erreurs au moment du développement
- Mécanisme de protection des routes intégré, utile pour séparer les accès selon le rôle utilisateur
- Bien adapté à des interfaces d'administration complexes comme le tableau de bord RI

Limites :

- Pas de rendu côté serveur sans Angular Universal, qui nécessite une configuration supplémentaire non négligeable
- Angular demande un temps de prise en main important : son architecture impose de nombreux concepts spécifiques (modules, décorateurs, injection de dépendances) qui ralentissent le démarrage
- Moins d'exemples disponibles que React/Next.js pour l'intégration avec le SSO CAS utilisé par Toulouse INP

4.1.2.3 Next.js 14

Next.js est un framework basé sur React qui permet de combiner rendu côté serveur et rendu côté client selon les besoins de chaque page [11].

Avantages :

- Rendu hybride : pages sensibles rendues côté serveur, composants interactifs côté client
 - les deux coexistent dans le même projet
- Middleware intégré pour bloquer l'accès aux pages selon le rôle utilisateur avant même leur chargement
- Appels à l'API Django réalisables côté serveur, sans exposer les tokens dans le navigateur
- App Router et React Server Components : chaque composant est rendu côté serveur par défaut, avec des layouts partagés par espace (RI, étudiant) qui structurent naturellement la séparation des interfaces
- TypeScript, Tailwind CSS et intégration SSO CAS/OIDC bien documentés

Limites :

- Configuration initiale un peu plus complexe qu'une application monopage classique

4.1.3 Synthèse comparative

Le tableau 4.1 récapitule les critères évalués pour chacune des trois solutions, afin de faciliter la comparaison et de justifier le choix retenu.

Critère	React + Vite	Angular	Next.js 14
Rendu côté serveur (SSR)	✗	Via Angular Universal (config requise)	✓
Protection des routes par rôle	Manuelle	Mécanisme intégré	Middleware intégré
Données sensibles côté serveur	✗	✗	✓
TypeScript natif	Optionnel	✓	✓
Intégration SSO CAS/OIDC	✓	Peu d'exemples avec le SSO CAS de Toulouse INP	✓
Rendu hybride SSR + CSR	✗	✗	✓

TABLE 4.1 – Comparaison des frameworks frontend

4.1.4 Décision

Next.js 14 est retenu. Son modèle de React Server Components permet de gérer naturellement les deux espaces du projet : les pages sensibles du tableau de bord RI sont rendues côté serveur avant toute vérification de rôle, tandis que les composants interactifs de l'espace étudiant s'exécutent côté client. C'est le seul framework parmi ceux évalués qui couvre ces deux cas dans le même projet, sans configuration supplémentaire complexe.

4.2 Choix du backend

Cette section présente les besoins spécifiques du backend, évalue les alternatives disponibles, puis justifie le choix retenu.

4.2.1 Besoins spécifiques

Le backend de ce projet ne se limite pas à une simple gestion de données. Il doit orchestrer de nombreuses entités métier liées entre elles et gérer des processus parfois complexes, comme le suivi des candidatures, des affectations ou des mobilités étudiantes.

L'application doit également assurer plusieurs fonctions essentielles : gérer les droits d'accès selon les profils (étudiants et administrateurs RI), conserver une trace des actions importantes via un journal d'audit, accompagner l'évolution de la base de données pendant

le développement, et s'intégrer au système d'authentification institutionnel CAS/OIDC de Toulouse INP.

Dans ce contexte, le choix du framework ne repose pas uniquement sur les performances, mais aussi sur la qualité des outils qu'il apporte pour structurer, sécuriser et faire évoluer l'application.

4.2.2 Choix du framework backend

4.2.2.1 Alternatives évaluées

Trois frameworks ont été évalués en regard des besoins identifiés précédemment.

4.2.2.1.1 Node.js

Environnement d'exécution JavaScript côté serveur, reconnu pour ses performances sur les applications qui traitent beaucoup de requêtes simultanées [12].

Avantages :

- Bonnes performances sur les traitements non bloquants
- Grand nombre de bibliothèques disponibles
- Gestion native des opérations asynchrones

Limites :

- Aucun outil natif pour la base de données, l'administration ou la gestion des droits : il faut tout assembler manuellement à partir de bibliothèques séparées, ce qui alourdit la configuration et la maintenance.
- Il n'existe pas d'outils suffisamment aboutis pour gérer des mécanismes avancés comme les machines à états ou la traçabilité fine des modifications, pourtant indispensables dans ce projet.
- L'utilisation de Node.js implique de travailler en JavaScript, alors que le reste du système est conçu en Python, ce qui introduit une complexité supplémentaire en termes de cohérence et de maintenance.
- La gestion des évolutions du schéma de base de données au fil des sprints demande davantage d'efforts d'organisation et de suivi, car elle n'est pas aussi intégrée nativement dans l'écosystème.

4.2.2.1.2 FastAPI

Framework Python récent, conçu pour construire des APIs performantes avec une validation automatique des données [13].

Avantages :

- Très bonnes performances grâce au support natif des requêtes asynchrones
- Validation automatique des données entrantes

- Documentation de l'API générée automatiquement

Limites :

- Pas d'outil intégré pour la base de données, l'administration ou la gestion des droits
- Les cycles de vie des entités, le journal d'audit et les permissions seraient à construire de zéro
- L'outil de migration de schéma demande davantage de configuration et d'intervention manuelle, ce qui peut ralentir les cycles d'itération en développement.
- FastAPI est surtout adapté à des APIs légères et rapides, et moins à des applications contenant beaucoup de logique métier.

4.2.2.1.3 Django

Framework Python complet, pensé pour développer rapidement des applications web avec beaucoup de logique interne [14].

Avantages :

- Gestion de la base de données, authentification, permissions et interface d'administration fournis nativement, sans aucune configuration supplémentaire
- `django-fsm` permet de définir des cycles de vie sécurisés sur chaque entité, avec une option qui empêche toute modification directe du statut en dehors des transitions autorisées
- `django-auditlog` enregistre automatiquement toutes les modifications avec l'auteur et la date — indispensable pour les décisions réglementaires
- La gestion des évolutions de schéma est intégrée nativement, ce qui facilite les changements à chaque sprint
- S'intègre facilement avec le SSO CAS/OIDC via des bibliothèques existantes et maintenues

Limites :

- Django était historiquement limité sur les traitements asynchrones, mais cette lacune a été corrigée depuis la version 5
- Légèrement moins performant que FastAPI sur des calculs très complexes, ce qui peut être contourné en passant par des requêtes SQL directes pour les agrégations de durées CTI

4.2.2.2 Synthèse comparative

Le tableau 4.2 récapitule les critères évalués pour chacune des trois solutions, afin de faciliter la comparaison et de justifier le choix retenu.

Critère	Node.js	FastAPI	Django 5
ORM et gestion BDD intégrés	✗	✗	✓
Interface d'administration	✗	✗	✓
Machines à états (FSM)	✗	✗	✓
Migrations de schéma	Manuel	nécessite config	✓
Authentification SSO	Manuel	Manuel	✓
Journal d'audit automatique	Manuel	Manuel	✓
Support ASGI / async	✓	✓	✓

TABLE 4.2 – Comparaison des frameworks backend

4.2.2.3 Décision

Django 5 est retenu car le projet repose sur de nombreuses entités liées entre elles, avec des cycles de vie distincts, un journal d'audit obligatoire et une gestion fine des droits entre deux profils utilisateurs.

Utiliser une autre solution aurait impliqué de reconstruire une grande partie de ces mécanismes.

Django permet de disposer de ces fonctionnalités directement, ce qui est un avantage important dans ce projet.

4.2.3 Choix de la couche API

Une fois le framework backend retenu, il reste à choisir la bibliothèque la mieux adaptée pour construire l'API. Cette section évalue les deux solutions compatibles avec Django et justifie le choix retenu.

4.2.3.1 Besoins spécifiques

L'API doit permettre les échanges entre Next.js et Django, valider des structures de données parfois complexes (vœux ordonnés, quotas par département, rapports d'import), générer automatiquement une documentation utilisable par le développeur frontend, et répondre de façon non bloquante aux requêtes qui déclenchent des traitements longs (lancement de l'algorithme, recalcul).

4.2.3.2 Alternatives évaluées

Deux solutions ont été évaluées pour exposer l'API entre le frontend Next.js et le backend Django.

4.2.3.2.1 Django REST Framework (DRF)

Django REST Framework est la bibliothèque la plus utilisée pour construire des APIs avec

Django. Elle fournit un ensemble complet d'outils pour la sérialisation, la validation et la gestion des permissions [15].

Avantages :

- Très mature et bien documenté, avec une grande communauté
- Outillage complet : gestion des tokens JWT, filtres, pagination, génération de documentation via `drf-spectacular`
- Fonctionne parfaitement avec l'ORM et les permissions Django

Limites :

- Le support des requêtes asynchrones est partiel, ce qui pose problème pour les endpoints qui déclenchent le lancement de l'algorithme ou les synchronisations avec MoveON, Pégase et Eudonet
- Les sérialiseurs DRF demandent beaucoup de code répétitif, surtout quand on travaille avec plusieurs entités
- La documentation de l'API nécessite une configuration externe séparée, à maintenir en parallèle

4.2.3.2.2 Django Ninja

Framework API pour Django, basé sur une validation des données structurée et typée via Pydantic [16].

Avantages :

- Toutes les routes supportent nativement les traitements asynchrones : un endpoint peut lancer un calcul ou déclencher une synchronisation sans bloquer le serveur, en déléguant l'exécution à un système de traitement en arrière-plan.
- La validation des données via Pydantic est plus concise que DRF
- La documentation de l'API est générée automatiquement, sans aucune configuration supplémentaire
- Conserve entièrement l'écosystème Django : ORM, administration, cycles de vie, permissions

Limites :

- Plus récent que DRF, avec moins d'extensions disponibles
- Moins de ressources en ligne pour des cas très spécifiques

4.2.3.3 Synthèse comparative

Le tableau 4.3 récapitule les critères évalués pour chacune des deux solutions, afin de faciliter la comparaison et de justifier le choix retenu.

Critère	DRF	Django Ninja
Support async natif	Non natif, configuration requise	✓
Validation Pydantic (typage strict)	✗	✓
Documentation auto-générée	✗	✓
Intégration ORM Django	✓	✓
Maturité et communauté	✓	Framework récent, moins de ressources
Réduction du code répétitif	✗	✓

TABLE 4.3 – Comparaison DRF et Django Ninja

4.2.3.4 Décision

Django Ninja est retenu. Les endpoints les plus critiques du projet — déclenchement de l’algorithme, recalcul, synchronisations ETL — doivent répondre immédiatement sans bloquer l’interface pendant le traitement. Django Ninja gère cela nativement, là où DRF aurait demandé des contournements. La validation Pydantic réduit aussi significativement le code répétitif sur plusieurs entités.

La stack backend retenue est donc : **Django 5 + Django Ninja**.

4.3 Choix de la base de données

Cette section présente les besoins spécifiques liés au stockage des données, évalue les alternatives disponibles, puis justifie le choix retenu.

4.3.1 Besoins spécifiques

La base de données doit supporter un modèle avec plusieurs entités reliées entre elles par de nombreuses clés étrangères, stocker les réponses brutes des APIs externes avant transformation (**RawImport**), et garantir la cohérence des données lors des opérations qui touchent plusieurs entités à la fois (transitions de cycle de vie, affectations, journal d’audit).

4.3.2 Alternatives évaluées

Trois systèmes de gestion de base de données ont été évalués en regard des besoins identifiés précédemment.

4.3.2.1 MySQL

MySQL est un système de gestion de base de données relationnelle très répandu, bien supporté par l’écosystème Django et largement utilisé en production [17].

Avantages :

- Système très répandu, avec une grande communauté et une documentation abondante
- Bonne intégration avec l'écosystème Django
- Largement utilisé en production, avec des outils de sauvegarde matures

Limites :

- Support moins adapté pour le stockage et l'exploitation de données au format JSON, notamment pour les imports bruts (`RawImport`)
- Fonctionnalités limitées pour gérer des mécanismes de file d'attente de tâches
- Moins adapté aux opérations transactionnelles complexes impliquant plusieurs entités liées

4.3.2.2 MongoDB

MongoDB est une base de données orientée documents, conçue pour offrir une grande flexibilité dans la structure des données et particulièrement adaptée au stockage JSON [18].

Avantages :

- Grande flexibilité dans la structure des données
- Adapté au stockage de documents JSON
- Bonnes performances sur les lectures de documents non structurés

Limites :

- Le modèle de données du projet repose fortement sur des relations entre entités (quotas, affectations, mobilités), ce qui rend l'absence de relations structurées moins adaptée
- Intégration limitée avec les outils ORM utilisés dans le projet
- Absence de support direct pour certains mécanismes de traitement asynchrone utilisés dans l'architecture

4.3.2.3 PostgreSQL 16

PostgreSQL est un système de gestion de base de données relationnelle open source, reconnu pour sa robustesse, la richesse de ses fonctionnalités et son respect strict des standards SQL [19].

Avantages :

- Gestion robuste des transactions, adaptée aux opérations impliquant plusieurs entités liées
- Support natif du format JSON avec indexation, utile pour le stockage des données brutes (`RawImport`)

- Possibilité d'utiliser la base comme support de file d'attente pour les traitements asynchrones
- Bon niveau d'intégration avec les outils de développement du projet
- Outils de sauvegarde et de restauration fiables

Limites :

- Mise en place initiale plus complète, nécessitant une configuration structurée dès le début du projet

4.3.3 Synthèse comparative

Le tableau 4.4 récapitule les critères évalués pour chacune des trois solutions, afin de faciliter la comparaison et de justifier le choix retenu.

Critère	MySQL / MariaDB	MongoDB	PostgreSQL 16
Transactions ACID multi-tables	✓	ACID ajouté en v4.0, moins fiable qu'un SGBD relationnel	✓
Support JSONB natif + indexation	JSON simple, pas d'indexation avancée	✓	✓
File d'attente de tâches	✗	✗	✓
Intégration ORM Django	✓	Support via PyMongo sans migration Django	✓
Extension pgcrypto (chiffrement)	✗	✗	✓
Outils de sauvegarde fiables	✓	✓	✓

TABLE 4.4 – Comparaison des systèmes de gestion de base de données

4.3.4 Décision

PostgreSQL 16 est retenu. Il répond de manière cohérente à l'ensemble des besoins identifiés — gestion des relations, support du JSON natif, traitements asynchrones et chiffrement — dans un seul service, sans introduire de composant supplémentaire dans l'infrastructure.

4.4 Choix du format de stockage des imports bruts

Cette section précise comment sont stockées les réponses brutes des APIs externes avant leur traitement, en évaluant les alternatives disponibles et en justifiant le format retenu.

4.4.1 Besoins spécifiques

L'entité `RawImport` doit conserver la réponse complète de chaque appel d'API externe (MoveON, Pégase, Eudonet) avant toute transformation. Ces données sont semi-structurées et variables d'une source à l'autre. Le format retenu doit permettre de relire, d'interroger et de tracer ces imports tout en restant cohérent avec le reste du modèle de données.

4.4.2 Alternatives évaluées

4.4.2.1 Fichiers plats (JSON sur le système de fichiers)

Un fichier plat est un fichier texte brut qui stocke des données sans structure hiérarchique ni mécanisme d'indexation. Dans ce contexte, chaque réponse d'API serait sérialisée en JSON et enregistrée dans un répertoire dédié sur le serveur, un fichier par import.

Avantages :

- Mise en place immédiate, sans configuration particulière
- Lecture directe par n'importe quel éditeur de texte

Limites :

- Aucune possibilité d'interroger le contenu avec des requêtes SQL
- Intégrité référentielle avec les autres entités non garantie
- Gestion des accès concurrents et des transactions non assurée
- Sauvegarde dissociée du reste de la base de données

4.4.2.2 Base NoSQL dédiée (MongoDB)

Les imports bruts sont stockés dans une base documentaire séparée, spécialisée dans le stockage de documents JSON.

Avantages :

- Très bien adapté au stockage de documents JSON hétérogènes
- Bonne performance sur les lectures de documents non structurés

Limites :

- Service supplémentaire à déployer, superviser et sauvegarder
- Aucune relation possible avec les entités de la base relationnelle
- Surcoût d'infrastructure injustifié pour ce seul cas d'usage

4.4.2.3 Type JSONB dans PostgreSQL

Les réponses brutes sont stockées dans une colonne de type **JSONB** au sein de la table **RawImport**, dans la même base PostgreSQL.

Avantages :

- Stockage dans la base existante, sans service supplémentaire
- Indexation du contenu JSON pour des requêtes efficaces
- Jointures possibles avec les autres tables du modèle
- Mêmes garanties ACID et même périmètre de sauvegarde que le reste

Limites :

- Moins flexible que MongoDB pour des structures JSON très hétérogènes, ce qui reste sans impact ici compte tenu du faible nombre de sources

4.4.3 Synthèse comparative

Critère	Fichiers plats	MongoDB	JSONB (PostgreSQL)
Requêtes SQL sur le contenu	✗	✗	✓
Intégrité référentielle	✗	✗	✓
Garanties ACID	✗	Transactions ACID ajoutées en v4.0, moins fiables que PostgreSQL	✓
Aucun service supplémentaire	✓	✗	✓
Indexation du contenu JSON	✗	✓	✓
Sauvegarde unifiée	✗	✗	✓

TABLE 4.5 – Comparaison des formats de stockage pour **RawImport**

4.4.4 Décision

Le type **JSONB** de PostgreSQL est retenu pour le stockage des imports bruts. Il évite toute fragmentation de l'infrastructure en réutilisant la base déjà en place, tout en offrant les capacités d'interrogation nécessaires sur les données semi-structurées. Les fichiers plats ne permettent pas de garantir la cohérence avec le reste du modèle, et l'ajout de MongoDB n'apporterait aucun avantage fonctionnel pour ce seul cas d'usage.

4.5 Choix du gestionnaire de tâches asynchrones

Cette section présente les besoins en traitement asynchrone du projet, évalue les solutions disponibles, puis justifie le choix retenu.

4.5.1 Besoins spécifiques

Certaines opérations du système ne peuvent pas s'exécuter de façon synchrone sans bloquer l'interface utilisateur. Deux catégories de tâches ont été identifiées.

La première catégorie regroupe les tâches déclenchées par des événements : le lancement de l'algorithme d'affectation, le recalcul des durées après modification d'un état, et le traitement des imports qui peuvent durer plusieurs secondes.

La seconde catégorie regroupe les tâches planifiées : les synchronisations nocturnes avec MoveON, Pégase et Eudonet, ainsi que la suppression automatique des imports bruts anciens.

Le volume reste modéré — quelques dizaines de tâches par jour en fonctionnement normal, quelques centaines lors des périodes de traitement — sans besoin de traitement intensif en continu.

4.5.2 Alternatives évaluées

Deux solutions ont été évaluées pour la gestion des tâches asynchrones dans l'écosystème Django.

4.5.2.1 Celery + Redis

Celery est la bibliothèque Python de référence pour l'exécution de tâches en arrière-plan. Elle s'appuie sur un broker de messages externe — généralement Redis ou RabbitMQ — pour acheminer les tâches vers les workers [20] [21].

Avantages :

- Très mature, bien documenté, avec une grande communauté
- Capable de traiter un très grand volume de tâches en parallèle
- Compatible avec plusieurs brokers (Redis, RabbitMQ)
- Outil de supervision dédié (Flower)

Limites :

- Trois services supplémentaires à déployer et maintenir : le worker Celery, le planificateur Celery Beat et Redis
- Redis consomme entre 50 et 200 Mo de mémoire pour un rôle que PostgreSQL, déjà présent, peut assurer seul
- Complexité disproportionnée par rapport au volume réel du projet

4.5.2.2 Django-Q2 + PostgreSQL

Django-Q2 est une bibliothèque de gestion de tâches asynchrones conçue spécifiquement pour Django. Elle utilise directement PostgreSQL comme file d'attente, sans nécessiter de broker externe [22].

Avantages :

- Aucun service supplémentaire : PostgreSQL joue le rôle de file d'attente sans configuration additionnelle
- Planificateur de tâches intégré, adapté aux synchronisations nocturnes et aux suppressions programmées
- Supervision accessible depuis l'interface d'administration Django existante, sans outils tiers
- Seulement deux services à opérer au lieu de cinq avec Celery

Limites :

- Moins performant que Celery pour des volumes très élevés, sans incidence pour ce projet
- Communauté plus restreinte que Celery

4.5.3 Synthèse comparative

Le tableau 4.6 récapitule les critères évalués pour chacune des deux solutions, afin de faciliter la comparaison et de justifier le choix retenu.

Critère	Celery + Redis	Django-Q2 + PostgreSQL
Aucun service supplémentaire	✗	✓
Planificateur intégré	Celery Beat disponible mais service séparé	✓
Supervision via admin Django	✗	✓
Adapté au volume du projet	Surdimensionné pour ce volume de tâches	✓
Maturité et communauté	✓	Communauté plus restreinte que Celery
Intégration native avec Django	Nécessite une configuration supplémentaire	✓

TABLE 4.6 – Comparaison des gestionnaires de tâches asynchrones

4.5.4 Décision

Django-Q2 avec PostgreSQL est retenu. Il réutilise la base de données déjà en place comme file d'attente, ce qui évite d'introduire un service supplémentaire dans l'infrastructure. Le volume de tâches du projet ne justifie pas la complexité opérationnelle de Celery. La supervision des traitements depuis l'interface d'administration Django existante simplifie également le suivi quotidien pour le service RI.

4.6 Choix du stockage de fichiers

Cette section présente les besoins en stockage de fichiers du projet, évalue les solutions disponibles, puis justifie le choix retenu.

4.6.1 Besoins spécifiques

Le projet requiert un mécanisme de dépôt et de stockage de fichiers pour les justificatifs soumis par les étudiants lors de la déclaration de mobilités complémentaires. Ces fichiers doivent être stockés de manière sécurisée et chiffrée, accessibles uniquement par le service RI et l'étudiant concerné, avec une suppression automatique conforme au RGPD.

4.6.2 Alternatives évaluées

Deux approches ont été évaluées pour répondre à ces besoins.

4.6.2.1 Système de fichiers local

Le stockage local consiste à enregistrer les fichiers directement sur le système de fichiers du serveur hébergeant l'application Django, dans un répertoire dédié.

Avantages :

- Simple à configurer, sans dépendance supplémentaire

Limites :

- Aucun chiffrement natif des fichiers stockés
- Non portable entre environnements (développement, staging, production)
- Difficile à sauvegarder et à migrer indépendamment de la base de données

4.6.2.2 MinIO (stockage objet S3-compatible)

MinIO est une solution de stockage objet open source, auto-hébergeable et compatible avec l'API S3 d'Amazon [23]. Elle s'intègre nativement dans une infrastructure Docker et ne nécessite aucun service cloud externe.

Avantages :

- Stockage objet auto-hébergé, compatible avec l’API S3 standard
- Chiffrement côté serveur (SSE-S3) des fichiers stockés
- Génération d’URLs pré-signées pour un accès temporaire et sécurisé, sans exposer les fichiers directement
- Déployable comme container Docker interne, sans aucune dépendance externe ni service cloud
- Politique de rétention et suppression automatique configurables, conformes aux exigences RGPD

Limites :

- Configuration initiale plus complexe qu’un stockage local

4.6.3 Synthèse comparative

Le tableau 4.7 récapitule les critères évalués pour chacune des deux solutions, afin de faciliter la comparaison et de justifier le choix retenu.

Critère	Fichiers local	MinIO
Chiffrement natif (SSE-S3)	✗	✓
URLs pré-signées (accès sécurisé)	✗	✓
Portabilité multi-environnements	✗	✓
Suppression automatique (RGPD)	✗	✓
Déploiement Docker intégré	✗	✓
Simplicité de configuration	✓	Plus complexe qu’un stockage local

TABLE 4.7 – Comparaison des solutions de stockage de fichiers

4.6.4 Décision

MinIO est retenu. MinIO peut être déployé selon deux modes : **standalone** (instance unique sur un seul serveur) ou **distributed** (plusieurs nœuds pour la haute disponibilité et la tolérance aux pannes). Le mode distribué est destiné aux infrastructures multi-serveurs nécessitant une réplication des données.

Dans le cadre de ce projet, le déploiement cible un **serveur unique on-premise** au sein de l’établissement. Le mode standalone est donc retenu : il correspond exactement à cette topologie, sans surcharger l’infrastructure avec des mécanismes de distribution inutiles. MinIO répond ainsi directement aux exigences de sécurité (chiffrement SSE-S3) et de conformité RGPD (rétention et suppression automatiques), tout en s’intégrant de façon transparente dans Docker Compose sans aucun service externe.

4.7 Couche de routage des requêtes

Cette section présente les besoins en matière de routage réseau, évalue les deux approches disponibles, puis justifie le choix retenu.

4.7.1 Besoins spécifiques

Le système doit disposer d'un composant chargé de recevoir les requêtes HTTP entrantes, d'assurer la terminaison SSL, de router le trafic vers le frontend Next.js et l'API Django Ninja, et de servir directement les fichiers statiques sans les faire transiter par l'application. Ce composant sera administré par les équipes IT de Toulouse INP ; sa configuration doit donc être lisible et maintenable sans expertise Docker particulière.

4.7.2 Alternatives évaluées

Deux approches ont été évaluées pour assurer ce rôle.

4.7.2.1 Reverse proxy

Un reverse proxy est un composant qui se place entre les clients et les services internes. Il reçoit les requêtes HTTP entrantes, assure la terminaison SSL, les achemine vers le service approprié selon des règles de routage, et sert les fichiers statiques directement sans solliciter l'application.

Avantages :

- Composant léger, dédié au routage et à la terminaison SSL
- Configuration statique (Nginx) ou dynamique (Traefik) selon le besoin
- Bien adapté à une architecture monolithique sur un seul serveur
- Maturité et documentation abondantes pour les deux outils

Limites :

- Ne centralise pas l'authentification ni le contrôle de débit (ce qui n'est pas requis ici)

4.7.2.2 API Gateway

Une API Gateway est un composant qui, en plus du routage, centralise la gestion de l'authentification, du contrôle de débit et de la transformation des requêtes. Elle est conçue pour exposer plusieurs microservices à des clients externes variés.

Avantages :

- Gestion centralisée de l'authentification et du contrôle de débit
- Adapté aux architectures microservices exposées à l'externe

Limites :

- Complexité et surcoût injustifiés pour une architecture déployée sur un seul serveur
- Le projet consomme des APIs externes, il n'en expose pas à des clients variés — ce cas d'usage ne s'applique pas

4.7.3 Synthèse comparative

Le tableau 4.8 récapitule les critères évalués pour chacune des deux approches, afin de faciliter la comparaison et de justifier le choix retenu.

Critère	Reverse proxy	API Gateway
Adapté à un seul serveur	✓	✗
Terminaison SSL	✓	✓
Routage vers services internes	✓	✓
Gestion auth. centralisée	✗	✓
Complexité d'exploitation	Faible	Élevée
Pertinent pour ce projet	✓	✗

TABLE 4.8 – Comparaison des approches de routage des requêtes

4.7.4 Décision

Le **reverse proxy** est retenu. L'architecture est déployée sur un seul serveur et le système consomme des APIs externes sans en exposer à des clients variés. Une API Gateway introduirait une complexité sans aucun bénéfice fonctionnel dans ce contexte.

4.7.5 Choix de l'implémentation : Nginx vs Traefik

Une fois le reverse proxy retenu, il reste à choisir l'outil d'implémentation. Cette partie évalue les deux solutions les plus utilisées dans l'écosystème Docker et justifie le choix retenu.

4.7.5.1 Besoins spécifiques

L'outil retenu doit assurer la terminaison SSL, le routage vers le frontend Next.js et l'API Django Ninja, et la distribution des fichiers statiques. Sa configuration doit être lisible et auditable par les équipes IT de Toulouse INP qui administreront le serveur, sans expertise Docker avancée.

4.7.5.2 Alternatives évaluées

Deux outils open source ont été comparés pour implémenter le reverse proxy.

4.7.5.2.1 Nginx

Nginx est un serveur web et reverse proxy très répandu, reconnu pour sa stabilité et la lisibilité de ses fichiers de configuration [24].

Avantages :

- Très stable et bien documenté, avec un comportement prévisible
- Configuration dans des fichiers texte simples, lisibles et faciles à auditer par n'importe quel administrateur système
- Gestion SSL/TLS fiable, excellentes performances sur le routage des requêtes

Limites :

- La gestion des certificats SSL nécessite un outil externe (Certbot)
- Configuration manuelle à mettre à jour lors de changements de services

4.7.5.2.2 Traefik

Traefik est un reverse proxy conçu pour les environnements conteneurisés. Il se configure dynamiquement à partir des labels des conteneurs Docker actifs [25].

Avantages :

- Configuration automatique à partir des conteneurs Docker actifs
- Gestion native des certificats SSL via Let's Encrypt
- Interface de supervision intégrée

Limites :

- Configuration déclarative moins lisible pour des équipes IT non familières avec Docker
- Comportement moins prévisible dans un environnement institutionnel avec des contraintes réseau spécifiques

4.7.5.3 Synthèse comparative

Le tableau 4.9 récapitule les critères évalués pour chacun des deux outils, afin de faciliter la comparaison et de justifier le choix retenu.

Critère	Nginx	Traefik
Configuration lisible (fichiers texte)	✓	Labels Docker, moins lisible pour les équipes IT
Gestion SSL native	Nécessite Certbot pour les certificats	✓
Automatisation Docker	Configuration manuelle à chaque changement	✓
Prévisibilité en environnement institutionnel	✓	Moins prévisible avec des contraintes réseau institutionnelles
Maintenabilité par équipes IT générales	✓	✗
Maturité et documentation	✓	✓

TABLE 4.9 – Comparaison des outils de reverse proxy

4.7.5.4 Décision

Nginx est retenu. Le serveur sera administré par les équipes IT de Toulouse INP. Disposer d’une configuration statique dans des fichiers texte qu’un administrateur système peut lire et modifier sans expertise Docker est, dans ce contexte institutionnel, plus important que l’automatisation dynamique proposée par Traefik.

4.8 Synthèse des choix technologiques

TABLE 4.10 – Synthèse des choix technologiques

Composant	Technologie retenue	Justification principale
Méthodologie de développement	Scrum	Développement itératif et adaptation progressive aux besoins du service RI
Frontend	Next.js 14	Rendu hybride SSR/CSR, séparation sécurisée des espaces RI et étudiant
Backend framework	Django 5	ORM, administration, gestion des droits et logique métier intégrés
Couche API	Django Ninja	Support natif de l’asynchrone, validation Pydantic, documentation automatique

Composant	Technologie retenue	Justification principale
Base de données	PostgreSQL 16	Transactions robustes, support JSON et bonne intégration avec Django
Gestionnaire de tâches asynchrones	Django-Q2 + PostgreSQL	Aucun service supplémentaire et supervision intégrée à Django
Stockage fichiers	MinIO (S3-compatible)	Stockage objet auto-hébergé, chiffrement SSE-S3, conforme RGPD, intégré comme container Docker sans service externe
Reverse proxy	Nginx	Configuration simple, stable et adaptée à un environnement institutionnel
Conteneurisation	Docker + Docker Compose	Déploiement reproductible et simplification de l'environnement
Gestion des états métier	django-fsm	Transitions sécurisées et intégration directe avec Django ORM
Journalisation et audit	django-auditlog	Historique automatique des modifications et traçabilité
Authentification	CAS/OIDC	Exigence institutionnelle — SSO Toulouse INP avec détection automatique du rôle depuis le token

Conclusion

Ce chapitre a présenté les composants techniques qui forment le socle de la plateforme : le frontend, le backend, la base de données, le stockage, les tâches asynchrones et le reverse proxy. Chaque technologie a été sélectionnée en fonction des besoins concrets du projet, en privilégiant la cohérence de l'ensemble et la simplicité d'exploitation. Les choix algorithmiques et les stratégies de performance font l'objet du chapitre suivant.

Choix algorithmiques et stratégies de performance

Plan

Introduction	46
5.1 Stratégie de performance base de données	46
5.1.1 Contexte et enjeux	46
5.1.2 Alternatives évaluées	46
5.1.2.1 Partitionnement par année	46
5.1.2.2 Indexation ciblée	46
5.1.3 Synthèse comparative	47
5.1.4 Décision	47
5.2 Choix de l'algorithme d'affectation	47
5.2.1 Besoins spécifiques	48
5.2.2 Alternatives évaluées	48
5.2.2.1 Algorithme glouton	48
5.2.2.2 Gale-Shapley	48
5.2.3 Synthèse comparative	49
5.2.4 Décision	49
5.3 Choix de la méthode de répartition des quotas par département	50
5.3.1 Besoins spécifiques	50
5.3.2 Alternatives évaluées	50
5.3.2.1 Méthode Hamilton (plus grands restes)	50
5.3.2.2 Méthode Jefferson (plus grandes moyennes)	51
5.3.2.3 Méthode Webster	51
5.3.3 Synthèse comparative	51
5.3.4 Décision	52
5.4 Choix du moteur de recommandation	52
5.4.1 Besoins spécifiques	52
5.4.2 Alternatives évaluées	52
5.4.2.1 Approche basée sur des règles (rule-based)	53
5.4.2.2 Approche par similarité (collaborative filtering)	53
5.4.2.3 Approche par apprentissage supervisé	53
5.4.2.3.1 Régression logistique	53
5.4.2.3.2 XGBoost	54
5.4.3 Synthèse comparative	54

5.4.4	Décision	54
5.5	Choix de l'orchestration RAG	55
5.5.1	Besoins spécifiques	55
5.5.2	Modèle de langage	55
5.5.3	Alternatives évaluées	56
5.5.3.1	LangChain	56
5.5.3.2	LlamaIndex	56
5.5.3.3	Haystack	56
5.5.3.4	Approche directe (sans framework)	57
5.5.4	Synthèse comparative	57
5.5.5	Décision	58
5.6	Synthèse des choix	58
	Conclusion	58

Introduction

Ce chapitre présente les choix algorithmiques et les stratégies de performance qui structurent la logique métier de la plateforme. Pour chaque composant, les alternatives disponibles sont comparées sur la base de critères précis, puis la méthode retenue est justifiée.

5.1 Stratégie de performance base de données

Cette section présente les enjeux de performance liés à la croissance des données, évalue les deux approches disponibles, puis justifie la stratégie retenue.

5.1.1 Contexte et enjeux

Plusieurs tables du projet — mobilités, affectations, imports — grossissent d’une campagne à l’autre. Le système doit pouvoir les filtrer rapidement par année universitaire, que ce soit pour afficher le tableau de bord RI, calculer les durées CTI ou lancer l’algorithme d’affectation. Deux approches ont été évaluées pour répondre à cet enjeu.

5.1.2 Alternatives évaluées

5.1.2.1 Partitionnement par année

Le partitionnement consiste à découper physiquement une table en sous-tables, une par valeur de la colonne de partition — ici, par année universitaire. PostgreSQL ne lit alors que la partition concernée lors d’une requête filtrée.

Avantages :

- Réduction significative du volume scanné pour les requêtes filtrées par année
- Efficace lorsque les tables atteignent plusieurs millions de lignes

Limites :

- La colonne de partitionnement doit être incluse dans toutes les contraintes d’unicité, ce qui peut créer des conflits avec les migrations Django au fil des sprints
- Complexité supplémentaire du schéma pour un volume qui ne le justifie pas

5.1.2.2 Indexation ciblée

L’indexation ciblée consiste à créer des index sur les colonnes les plus utilisées dans les filtres, sans modifier la structure physique des tables.

Avantages :

- Aucune modification du schéma ni des migrations Django

- Index déclarés directement dans les modèles, visibles dans le code et versionnés avec lui
- Transparent pour l'ORM, sans configuration PostgreSQL séparée

Limites :

- Moins efficace que le partitionnement pour des volumes de plusieurs millions de lignes
- sans incidence ici

5.1.3 Synthèse comparative

Le tableau 5.1 récapitule les critères évalués pour chacune des deux approches, afin de faciliter la comparaison et de justifier la stratégie retenue.

Critère	Partitionnement	Indexation ciblée
Adapté au volume du projet	✗	✓
Compatible avec les migrations Django	Contraintes d'unicité incompatibles avec certaines migrations	✓
Déclaration dans les modèles	✗	✓
Aucune configuration PostgreSQL séparée	✗	✓
Efficacité à très grande échelle	✓	Moins efficace pour plusieurs millions de lignes

TABLE 5.1 – Comparaison des stratégies de performance base de données

5.1.4 Décision

L'**indexation ciblée** est retenue. L'ENSEEIH7 traite quelques centaines d'étudiants en mobilité par an. Même en projetant ce volume sur dix ans, les tables restent très loin du seuil où le partitionnement apporterait un gain mesurable. Des index sont créés sur `academic_year_id` et sur la combinaison (`academic_year_id`, `student_id`) pour les tables les plus sollicitées, directement dans les modèles Django concernés.

5.2 Choix de l'algorithme d'affectation

Cette section présente les contraintes auxquelles l'algorithme doit répondre, évalue les approches disponibles, puis justifie le choix retenu.

5.2.1 Besoins spécifiques

L'algorithme doit attribuer à chaque étudiant une destination parmi ses vœux, en respectant un ensemble de contraintes institutionnelles. Les résultats ont un impact réel sur les parcours : le service RI doit pouvoir justifier chaque affectation. L'équité entre candidats et la reproductibilité des résultats sont des exigences essentielles.

Le classement des étudiants repose principalement sur le GPA, complété par des règles institutionnelles. Les affectations doivent respecter les quotas par accord de mobilité et par département, les contraintes de financement, ainsi que la situation des étudiants étrangers dans le classement global. Chaque étudiant exprime jusqu'à trois vœux ordonnés ; l'algorithme traite les vœux dans l'ordre et recherche une affectation alternative en cas d'indisponibilité.

5.2.2 Alternatives évaluées

Deux approches algorithmiques ont été évaluées pour répondre à ces besoins.

5.2.2.1 Algorithme glouton

L'algorithme glouton parcourt les candidats dans l'ordre de leur GPA et affecte chacun à son meilleur vœu encore disponible, sans jamais revenir sur une décision déjà prise.

Avantages :

- Simple à implémenter et à expliquer
- Résultat prévisible, complexité $O(n^2)$

Limites :

- Le résultat dépend de l'ordre de traitement : deux étudiants avec des GPA très proches peuvent se retrouver dans des situations très différentes selon leur position dans la liste
- Une légère variation de l'ordre d'entrée peut produire des résultats différents, ce qui est incompatible avec l'exigence de reproductibilité

5.2.2.2 Gale-Shapley

L'algorithme de Gale-Shapley est un algorithme d'affectation stable qui procède par rondes successives de propositions et de rejets différés jusqu'à convergence. Il garantit qu'aucune paire étudiant-destination ne peut former une meilleure association que celle retenue à l'issue du processus [GaleShapley1962].

Avantages :

- Résultat stable : il est mathématiquement impossible qu'un étudiant et une destination forment une meilleure paire que celle retenue, quelles que soient les données
- Équitable : le résultat ne dépend pas de l'ordre de traitement, deux exécutions avec les mêmes données donnent toujours le même résultat

- Optimal pour les étudiants dans la variante **Student-Proposing** : chacun obtient la meilleure destination possible compte tenu de son classement et de ses vœux
- Complexité $O(n^2)$, tout à fait adaptée à quelques centaines d'étudiants

Limites :

- Légèrement plus complexe à implémenter que l'algorithme glouton

5.2.3 Synthèse comparative

Le tableau 5.2 récapitule les critères évalués pour chacune des deux approches, afin de faciliter la comparaison et de justifier le choix retenu.

Critère	Algo. glouton	Gale-Shapley
Stabilité des affectations	✗	✓
Reproductibilité	✗	✓
Indépendance à l'ordre de traitement	✗	✓
Optimalité pour les étudiants	Résultat non garanti selon l'ordre de traitement	✓
Explicabilité des résultats	Difficile à justifier si l'ordre a influencé le résultat	✓
Simplicité d'implémentation	✓	Plus complexe que l'algo glouton, bien documenté

TABLE 5.2 – Comparaison des algorithmes d'affectation

5.2.4 Décision

Gale-Shapley est retenu, dans sa variante **Student-Proposing** : ce sont les étudiants qui font des propositions aux destinations, et non l'inverse. Cette variante garantit que chaque étudiant obtient la meilleure destination possible compte tenu de son classement et de ses vœux — propriété dite d'*optimalité côté proposant*.

Les contraintes de quota sont vérifiées à deux niveaux simultanément à chaque proposition : un quota global par accord bilatéral (capacité de l'université partenaire) et un quota local par département pédagogique (places allouées au sein de l'ENSEEIH). Une destination est disponible seulement si les deux niveaux sont respectés ; sinon la proposition est rejetée et l'étudiant passe à son vœu suivant.

L'algorithme glouton est écarté car il est sensible à l'ordre de traitement et ne garantit pas la reproductibilité. Gale-Shapley assure un résultat stable, équitable et reproductible dans ce cadre multi-contraint.

5.3 Choix de la méthode de répartition des quotas par département

Cette section présente les besoins liés à la répartition des quotas entre départements, évalue les méthodes disponibles, puis justifie le choix retenu.

5.3.1 Besoins spécifiques

Chaque accord bilatéral fixe un nombre total de places disponibles. Ce nombre doit ensuite être distribué entre les départements pédagogiques, ce qui soulève une question qui n'a pas de réponse évidente : sur quelle base décider combien de places revient à chaque département ?

Une répartition basée uniquement sur les demandes de l'année en cours serait insuffisante : elle ne tient pas compte du fait qu'un département peut avoir été systématiquement défavorisé les années précédentes, ni du fait que certains accords voient leur nombre de places évoluer d'une campagne à l'autre. Sans intégrer cet historique, la répartition risque d'être perçue comme arbitraire et de creuser des déséquilibres entre départements au fil des années.

Le besoin est donc de disposer d'une méthode qui distribue les places de façon proportionnelle à la demande réelle, tout en restant simple à comprendre, transparente et vérifiable par le service RI. La méthode retenue doit également garantir que la somme des places attribuées aux départements est exactement égale au nombre de places de l'accord, sans qu'aucune place ne soit perdue lors des arrondis.

5.3.2 Alternatives évaluées

Trois méthodes de répartition proportionnelle ont été comparées. Toutes partent du même principe : chaque département reçoit une quote-part proportionnelle à son nombre d'étudiants. La différence porte sur la manière de distribuer les places entières lorsque la division produit des résultats décimaux.

5.3.2.1 Méthode Hamilton (plus grands restes)

La méthode Hamilton attribue d'abord à chaque département la partie entière de sa quote-part. Les places restantes sont ensuite données, une par une, aux départements ayant les fractions les plus élevées, jusqu'à épuiser le total disponible.

Avantages :

- Simple à comprendre et à expliquer à tous les acteurs
- Résultat facile à vérifier manuellement
- Respecte les proportions au plus près du total disponible

Limites :

- Un département dont la quote-part est inférieure à 1 ne reçoit aucune place garantie, même si sa fraction est élevée

5.3.2.2 Méthode Jefferson (plus grandes moyennes)

La méthode Jefferson procède de façon itérative : à chaque étape, la prochaine place est attribuée au département dont le rapport entre sa taille et le nombre de places déjà reçues est le plus élevé.

Avantages :

- Aucun arrondi explicite nécessaire
- Résultats stables et cohérents d'une campagne à l'autre

Limites :

- Avantage les grands départements et pénalise les petits
- Calcul itératif moins intuitif à vérifier manuellement

5.3.2.3 Méthode Webster

La méthode Webster est proche de Jefferson, mais elle utilise un arrondi standard (seuil à 0,5) pour décider si un département reçoit une place supplémentaire ou non.

Avantages :

- Plus équilibrée que Jefferson pour les petits départements
- Bien adaptée aux cas où peu de places sont à distribuer

Limites :

- Calcul moins intuitif qu'Hamilton, plus difficile à vérifier sans outil
- Résultats moins prévisibles dans les cas limites

5.3.3 Synthèse comparative

Le tableau 5.3 récapitule les critères évalués pour chacune des trois méthodes, afin de faciliter la comparaison et de justifier le choix retenu.

Critère	Hamilton	Jefferson	Webster
Simplicité de calcul	✓	✗	✗
Vérification manuelle possible	✓	Calcul itératif difficile à suivre	Moins prévisible dans les cas limites
Équité entre départements	✓	Favorise les grands départements	✓
Respect du total disponible	✓	✓	✓
Adapté à un petit nombre de places	✓	✗	✓
Transparence pour les acteurs	✓	✗	✗

TABLE 5.3 – Comparaison des méthodes de répartition des quotas

5.3.4 Décision

La **méthode Hamilton** est retenue. Elle offre le meilleur équilibre entre équité, simplicité et transparence dans ce contexte. Les résultats peuvent être vérifiés à la main par le service RI, ce qui est important pour un processus qui touche directement les chances de départ des étudiants.

La méthode Jefferson est écartée car elle avantage systématiquement les grands départements, ce qui irait à l'encontre de l'équité visée. Webster est plus difficile à vérifier sans outil de calcul. Hamilton est donc la méthode la plus adaptée pour une répartition claire et justifiable.

5.4 Choix du moteur de recommandation

Cette section présente les besoins du module de recommandation, évalue les approches disponibles, puis justifie la méthode retenue.

5.4.1 Besoins spécifiques

Le moteur de recommandation a pour objectif d'aider les étudiants à choisir leurs destinations avant la saisie des vœux dans MoveON. Il ne prend pas de décision, mais propose un classement des destinations selon leur pertinence pour le profil de l'étudiant. Les recommandations doivent s'appuyer sur le profil académique et sur les données historiques disponibles, tout en restant interprétables pour l'étudiant.

5.4.2 Alternatives évaluées

Trois grandes familles d'approches ont été évaluées pour répondre à ce besoin.

5.4.2.1 Approche basée sur des règles (rule-based)

Cette approche repose sur des règles métier simples définies manuellement, telles que filtrer les destinations selon le département et classer par taux d'acceptation historique.

Avantages :

- Simple à mettre en place, sans données d'apprentissage
- Facile à comprendre et à expliquer

Limites :

- Peu flexible, ne s'adapte pas aux profils individuels
- Résultats souvent trop génériques

5.4.2.2 Approche par similarité (collaborative filtering)

Cette approche propose des destinations en se basant sur les choix d'étudiants ayant des profils similaires dans les promotions précédentes.

Avantages :

- Prend en compte les expériences passées des promotions
- Permet des recommandations plus personnalisées

Limites :

- Nécessite un volume important de données historiques
- Moins efficace pour les nouvelles destinations
- Mise en œuvre plus complexe

5.4.2.3 Approche par apprentissage supervisé

Cette approche consiste à entraîner un modèle pour prédire une probabilité de compatibilité entre un étudiant et une destination. Deux modèles ont été comparés au sein de cette famille.

5.4.2.3.1 Régression logistique

La régression logistique est un modèle statistique qui estime la probabilité d'un résultat à partir de variables d'entrée numériques. Elle est particulièrement adaptée aux volumes de données modérés [26].

Avantages :

- Modèle simple et rapide à entraîner
- Résultats faciles à interpréter et à expliquer
- Adapté à des volumes de données modérés

Limites :

- Capture difficilement des relations complexes entre variables

5.4.2.3.2 XGBoost

XGBoost est un algorithme de gradient boosting qui construit un ensemble d'arbres de décision de façon séquentielle pour améliorer la précision prédictive [26].

Avantages :

- Capable de modéliser des relations complexes entre variables
- Très bonnes performances prédictives

Limites :

- Plus complexe à entraîner et à maintenir
- Résultats moins interprétables
- Risque de surapprentissage avec un volume de données limité

5.4.3 Synthèse comparative

Le tableau 5.4 récapitule les critères évalués pour chacune des approches, afin de faciliter la comparaison et de justifier le choix retenu.

Critère	Règles	Collab.	Régression log.	XGBoost
Personnalisation	✗	✓	✓	✓
Volume de données requis	✓	✗	Efficace à partir de 500 observations	Risque de surapprentissage avec peu de données
Interprétabilité	✓	Difficile sans connaître les profils similaires	✓	✗
Risque de surapprentissage	✓	Dépend de la qualité des données historiques	✓	✗
Simplicité de maintenance	✓	✗	✓	✗

TABLE 5.4 – Comparaison des approches de recommandation

5.4.4 Décision

L'apprentissage supervisé par régression logistique est retenu. Le modèle estime, pour chaque couple étudiant–destination, une probabilité de compatibilité à partir des variables suivantes :

Variable	Description
GPA de l'étudiant	Note académique normalisée, principal signal de classement
Département pédagogique	Filtrage sur les accords ouverts au département de l'étudiant
Destination	Identifiant de l'université partenaire
Historique des affectations	Taux de sélection par destination sur les trois dernières années

TABLE 5.5 – Variables d'entrée du moteur de recommandation

Les destinations sont classées par ordre de probabilité décroissante afin de proposer une liste de recommandations personnalisées.

La régression logistique est retenue car elle offre un bon compromis entre simplicité, interprétabilité et facilité de maintenance. Le volume de données — environ 900 observations sur trois ans — ne justifie pas XGBoost, dont les performances supplémentaires s'accompagnent d'un risque accru de surapprentissage.

Risque de démarrage à froid. Certaines destinations rares ou récentes disposent de peu d'observations historiques. Pour ces cas, le modèle se rabat sur le GPA et le département comme seuls signaux, ce qui reste cohérent avec les règles de sélection. Le module sera activé à partir de la deuxième année de fonctionnement, une fois qu'un historique minimal d'une campagne complète aura été constitué.

5.5 Choix de l'orchestration RAG

Cette section présente les besoins du module RAG et le modèle de langage imposé, évalue les frameworks d'orchestration disponibles, puis justifie le choix retenu.

5.5.1 Besoins spécifiques

Le composant RAG du projet est un chatbot destiné aux étudiants, capable de répondre à des questions sur les procédures de mobilité à partir de la documentation institutionnelle de l'ENSEEIH : guides, critères d'éligibilité, délais et démarches administratives. Le cas d'usage est relativement simple : une base documentaire stable, des questions en français ou en anglais, et des réponses ancrées dans les documents officiels.

5.5.2 Modèle de langage

Le modèle de langage retenu est **Mistral** [27] (`mistral-small-latest`), conformément aux exigences de l'ENSEEIH. Il est accessible via l'**API Mistral ILAS**, une infrastructure nationale dédiée aux établissements de l'enseignement supérieur français, hébergée sur des serveurs localisés en France.

Ce mode d'accès institutionnel répond directement aux exigences RGPD : les données traitées restent dans l'infrastructure nationale et ne sont pas transmises à des tiers commerciaux. Il est néanmoins impératif que les données personnelles des étudiants ne soient jamais incluses dans les requêtes envoyées au modèle — les questions au chatbot sont transmises telles que saisies, sans enrichissement avec des données de dossier.

5.5.3 Alternatives évaluées

Quatre approches ont été évaluées pour orchestrer le pipeline RAG autour du modèle Mistral.

5.5.3.1 LangChain

LangChain est un framework d'orchestration IA généraliste, compatible avec un large éventail de modèles et de bases vectorielles [28].

Avantages :

- Grande communauté et nombreuses ressources disponibles
- Intégration simple avec Mistral et ChromaDB
- Adapté aux pipelines IA complexes et multi-étapes

Limites :

- API qui évolue fréquemment, induisant des risques de rupture
- Nombreuses dépendances, surcoût pour un chatbot documentaire simple

5.5.3.2 LlamaIndex

LlamaIndex est un framework spécialisé dans la recherche et la génération à partir de documents structurés [29].

Avantages :

- Conçu spécifiquement pour les applications RAG documentaires
- Bonne intégration avec Mistral et les bases vectorielles
- API plus stable, prise en main plus directe

Limites :

- Moins adapté aux architectures IA multi-agents très complexes

5.5.3.3 Haystack

Haystack est un framework orienté production pour les systèmes de recherche et de génération augmentée [30].

Avantages :

- Architecture modulaire adaptée aux déploiements en production
- Compatible avec Mistral

Limites :

- Documentation moins accessible pour des cas d’usage simples
- Intégration avec Mistral moins directe que LlamaIndex

5.5.3.4 Approche directe (sans framework)

L’approche directe consiste à implémenter manuellement le pipeline RAG via des appels à l’API Mistral, sans couche d’abstraction intermédiaire.

Avantages :

- Contrôle total sur chaque étape du pipeline
- Peu de dépendances externes

Limites :

- Nécessite de développer manuellement toute la logique RAG (chunking, indexation, retrieval, construction du prompt)
- Maintenance plus complexe à long terme

5.5.4 Synthèse comparative

Le tableau 5.6 récapitule les critères évalués pour chacune des quatre approches, afin de faciliter la comparaison et de justifier le choix retenu.

Critère	LangChain	LlamaIndex	Haystack	Direct
Spécialisé RAG	Généraliste, non optimisé pour le RAG	✓	Orienté pipelines complexes	✗
Intégration Mistral	✓	✓	Possible mais moins directe	✓
Stabilité de l’API	✗	✓	✓	✓
Adapté à un cas simple	Surcoût de dépendances pour un cas simple	✓	Trop complexe pour un chatbot documentaire	Logique RAG à développer entièrement
Maintenance à long terme	API instable, risque de rupture à chaque mise à jour	✓	✓	✗

TABLE 5.6 – Comparaison des frameworks d’orchestration RAG

5.5.5 Décision

LlamaIndex est retenu comme framework d'orchestration RAG, avec **ChromaDB** comme base vectorielle et **Mistral** comme modèle de langage. Cette combinaison correspond directement au besoin du projet : construire un chatbot documentaire simple, basé sur une base de connaissances stable. LlamaIndex fournit les fonctionnalités nécessaires sans surcoût de complexité, avec une API stable et une intégration directe avec Mistral et ChromaDB.

5.6 Synthèse des choix

TABLE 5.7 – Synthèse des choix algorithmiques et de performance

Composant	Méthode retenue	Justification principale
Stratégie de performance BDD	Indexation ciblée	Solution simple et suffisante pour le volume de données du projet
Algorithme d'affectation	Gale-Shapley	Stabilité, équité et reproductibilité des affectations
Répartition des quotas	Hamilton	Méthode proportionnelle simple, transparente et vérifiable manuellement
Moteur de recommandation	Régression logistique	Modèle simple, interprétable et adapté à un volume de données limité
Orchestration RAG	LlamaIndex + ChromaDB	Solution adaptée à un chatbot documentaire simple et stable
Modèle de langage	Mistral	Compatible avec le besoin multilingue du chatbot étudiant

Conclusion

Ce chapitre a présenté les choix algorithmiques et les stratégies de performance qui structurent la logique métier de la plateforme. Chaque méthode a été sélectionnée en fonction des besoins concrets du projet, en privilégiant l'équité, la reproductibilité et la simplicité d'exploitation. Ces choix définissent le comportement intelligent du système sur lequel repose le développement présenté dans les chapitres suivants.

Conception du système

Plan

Introduction	61
6.1 Product Backlog	61
6.1.1 Priorités MoSCoW	61
6.1.2 Story points	61
6.1.3 Tableau du Product Backlog	62
6.2 Planification des sprints	65
6.2.1 Diagramme de Gantt	66
6.3 Architecture logicielle	69
6.3.1 Vue d'ensemble	69
6.3.2 Couche présentation — Next.js 14	70
6.3.3 Couche applicatif — Django 5 + Django Ninja	70
6.3.4 Couche traitement asynchrone — Django-Q2	70
6.3.5 Couche données — PostgreSQL 16	70
6.3.6 Couche intégration — Adaptateurs ETL	71
6.3.6.1 Stratégie d'upsert	71
6.3.6.2 Clé de jointure : l'INE	72
6.3.6.3 Stratégie de synchronisation	72
6.3.7 Sécurité et contrôle d'accès	72
6.3.7.1 Flux d'authentification OIDC	72
6.3.7.2 Contrôle d'accès (RBAC)	73
6.3.7.3 Gestion du renouvellement des tokens JWT	73
6.3.7.4 Journal d'audit	73
6.3.7.5 Protection des données (RGPD)	74
6.4 Architecture physique et déploiement	74
6.4.1 Vue d'ensemble	74
6.4.2 Containers Docker	75
6.4.3 Flux réseau	76
6.4.4 Environnements	76
6.4.5 CI/CD	76
6.4.6 Stratégie de migration de schéma	77
6.4.7 Stratégie de rollback	77
6.4.8 Politique de sauvegarde	77
6.4.9 Gestion des secrets	78

6.5	Modèle de données	78
6.5.1	Organisation par packages	78
6.5.2	Principes structurants	79
6.5.2.1	L'année universitaire comme axe temporel	79
6.5.2.2	L'INE comme identifiant pivot	79
6.5.2.3	Hierarchie quotas INP / quotas N7	79
6.6	Machines à états finis (FSM)	79
6.6.1	Principe et implémentation	79
6.6.2	FSM de l'année universitaire	80
6.6.3	FSM des entités métier	81
6.7	Flux de données : de la source au workflow	81
6.7.1	Détail de l'étape de nettoyage et validation	82
6.7.1.1	Description du processus de transformation	82
6.7.1.1.1	Standardisation des formats	83
6.7.1.1.2	Normalisation sémantique	83
6.7.1.1.3	Gestion des valeurs manquantes	83
6.7.1.1.4	Détection et gestion des doublons	83
6.7.1.1.5	Validation métier	84
6.7.1.1.6	Traçabilité et gestion des erreurs	84
6.7.1.2	Architecture du pipeline de nettoyage	84
6.7.1.3	Technologies et outils utilisés	85
6.8	Gestion des risques	85
	Conclusion	87

Introduction

Ce chapitre présente la conception globale du système : le product backlog Scrum qui structure le développement, l’architecture logicielle et physique, le modèle de données et les machines à états qui gouvernent les cycles de vie métier.

6.1 Product Backlog

6.1.1 Priorités MoSCoW

Chaque user story est priorisée selon la méthode MoSCoW, qui permet de séparer l’essentiel de l’accessoire et d’éviter que des fonctionnalités secondaires ne retardent la livraison du cœur du système.

- **M — Must Have** : fonctionnalités sans lesquelles le système ne peut pas remplir son objectif. Leur absence rendrait le projet inutilisable pour le service RI.
- **S — Should Have** : fonctionnalités importantes, mais dont le report à une version suivante reste acceptable sans compromettre le MVP.
- **C — Could Have** : fonctionnalités souhaitables, planifiées uniquement si la capacité restante du sprint le permet une fois les priorités supérieures traitées.
- **W — Won’t Have** : fonctionnalités explicitement exclues du périmètre actuel, documentées pour ne pas créer d’attente non maîtrisée.

6.1.2 Story points

L’effort de chaque story est estimé en story points selon l’échelle de Fibonacci. Cette échelle non linéaire reflète le fait que l’incertitude croît avec la complexité : la différence entre 5 et 8 points n’est pas la même que celle entre 1 et 2. Le tableau 6.1 donne les correspondances retenues pour ce projet, avec des exemples concrets.

SP	Effort estimé	Exemples typiques
1	Très faible — quelques heures	Ajout d’un champ, endpoint simple
2	Faible — une demi-journée	Nouveau modèle Django, endpoint CRUD
3	Modéré — une journée	Service métier, FSM à 3 états
5	Significatif — deux jours	Pipeline d’import, interface multi-composants
8	Élevé — trois à quatre jours	Intégration SSO, import Excel avec rapport
13	Très élevé — une semaine	Algorithme d’affectation, moteur de recommandation
21	Exceptionnel — plus d’une semaine	Chatbot RAG complet

TABLE 6.1 – Échelle des story points

6.1.3 Tableau du Product Backlog

TABLE 6.2 – Product Backlog

ID	Titre	User Story	Prior.	SP	Sprint
Fondations techniques					
US-01	Référentiels de base	En tant qu' administrateur RI , je veux disposer d'un référentiel stable des pays, départements et universités partenaires (synchronisé depuis MoveON) afin que toutes les entités du système s'appuient sur des données de référence cohérentes.	M	5	S1
US-02	Année universitaire et FSM	En tant qu' administrateur RI , je veux que le cycle de vie de l'année universitaire soit géré par une machine à états (initialisation → recommandation → consolidation → pré-affectation → validation → clôturée) afin de contrôler chaque phase de la campagne de manière sécurisée.	M	4	S1
US-03	Accords et quotas	En tant qu' administrateur RI , je veux un modèle de données complet pour les accords et leurs quotas (INP global, estimation N7, ajustement N7, répartition par département), synchronisé depuis MoveON, afin d'avoir une base fiable pour l'algorithme d'affectation.	M	8	S1
Données et traçabilité					
US-04	Audit log et traçabilité	En tant qu' administrateur RI , je veux que chaque action importante (import, transition FSM, modification, validation) soit enregistrée avec l'auteur et l'horodatage, afin de garantir la traçabilité complète des décisions.	M	3	S2
US-05	Données étudiants	En tant qu' administrateur RI , je veux importer et synchroniser les données académiques des étudiants (INE, GPA, département, niveau) depuis Pégase, afin d'alimenter l'algorithme d'affectation avec des données étudiants fiables et à jour.	M	8	S2
US-06	Vœux étudiants	En tant qu' administrateur RI , je veux importer les dossiers de candidature et les vœux ordonnés des étudiants depuis MoveON, et les consulter par étudiant, afin de les intégrer dans le système d'affectation.	M	5	S2

ID	Titre	User Story	Prior.	SP	Sprint
US-07	Rapport d'import	En tant qu' administrateur RI , je veux consulter un rapport détaillé après chaque import (succès, erreurs, doublons) avec la raison précise de chaque rejet, afin de corriger les anomalies rapidement.	M	3	S2
Algorithme d'affectation					
US-08	Algorithme Gale-Shapley	En tant qu' administrateur RI , je veux lancer l'algorithme d'affectation (Deferred Acceptance) qui affecte chaque étudiant à son meilleur vœu disponible en respectant les quotas par département, afin de produire une proposition équitable et stable.	M	13	S3
US-09	Supervision des affectations	En tant qu' administrateur RI , je veux consulter les propositions d'affectation, les ajuster manuellement pour les cas particuliers et tracer chaque modification avec un motif, avant de valider et publier les résultats.	M	8	S3
US-10	Consultation des résultats	En tant qu' étudiant , je veux consulter mon affectation une fois les résultats publiés afin de connaître ma destination et les démarches à suivre.	M	3	S3
Sources complémentaires et mobilités					
US-11	Répartition des quotas par département	En tant qu' administrateur RI , je veux que le système calcule automatiquement la répartition des places de chaque accord entre les départements pédagogiques (SN, 3EA, MF2E) selon la méthode Hamilton (plus grands restes), afin que l'algorithme d'affectation dispose de quotas par département précis, équitables et vérifiables.	M	5	S4
US-12	Stages internationaux (Eudonet)	En tant qu' administrateur RI , je veux importer et synchroniser les conventions de stages internationaux depuis Eudonet afin de les comptabiliser dans le calcul CTI de chaque étudiant.	M	8	S4
US-13	Mobilités entrantes (Excel)	En tant qu' administrateur RI , je veux importer les mobilités entrantes via un template Excel standardisé afin de centraliser le suivi des étudiants internationaux accueillis à l'ENSEEIH.	M	8	S4

ID	Titre	User Story	Prior.	SP	Sprint
US-14	Réconciliation sans INE	En tant qu' administrateur RI , je veux que le système me propose des correspondances pour les enregistrements sans INE valide (via nom, prénom, date de naissance) afin de résoudre manuellement les cas ambigus.	S	5	S4
US-15	Déclaration mobilité complémentaire	En tant qu' étudiant , je veux déclarer une expérience internationale hors cursus (summer school, séjour linguistique, programme court) avec dépôt de justificatifs, afin qu'elle soit prise en compte dans mon calcul de durée CTI.	M	8	S4
US-16	Validation des mobilités complémentaires	En tant qu' administrateur RI , je veux consulter les justificatifs soumis, valider ou rejeter chaque demande avec un motif, afin de maintenir la qualité des données comptabilisées dans le calcul CTI.	M	5	S4
Conformité CTI					
US-17	Calcul automatique des durées CTI	En tant qu' administrateur RI , je veux que le système calcule automatiquement la durée totale de mobilité de chaque étudiant (séjours académiques, stages, expériences complémentaires validées) afin de vérifier la conformité CTI sans intervention manuelle.	M	8	S5
US-18	Indicateur CTI pour l'étudiant	En tant qu' étudiant , je veux visualiser ma durée cumulée de mobilité et sa comparaison avec le seuil CTI requis afin de savoir si je suis en règle pour l'obtention de mon diplôme.	M	3	S5
US-19	Export rapport CTI	En tant qu' administrateur RI , je veux exporter un rapport de conformité CTI pour l'ensemble des étudiants de la promotion afin de le soumettre lors des audits.	S	5	S5
Espace étudiant					
US-20	Catalogue des destinations	En tant qu' étudiant , je veux consulter les universités partenaires avec les places disponibles estimées, les conditions d'éligibilité (GPA minimum, département) et les informations pratiques, afin d'explorer les options avant de saisir mes vœux dans MoveON.	M	5	S5
US-21	Suivi de candidature	En tant qu' étudiant , je veux suivre le statut de ma candidature en temps réel (soumis, en traitement, affecté, rejeté) afin d'être informé de l'avancement sans solliciter le service RI.	M	3	S5

ID	Titre	User Story	Prior.	SP	Sprint
US-22	Gestion des mobilités entrantes	En tant qu' administrateur RI , je veux gérer le suivi administratif des étudiants internationaux accueillis (procédure envoyée, candidature Moodle, lettre d'acceptation, inscription) afin de centraliser le suivi dans un seul outil.	M	5	S5
Supervision et authentification					
US-23	Console de supervision de campagne	En tant qu' administrateur RI , je veux un tableau de bord affichant l'état en temps réel de la campagne (candidatures reçues, imports réalisés, anomalies en attente, avancement par phase) afin de piloter efficacement la campagne.	S	8	S5
US-24	Contrôle des prérequis FSM	En tant qu' administrateur RI , je veux que le système m'empêche de passer à la phase de consolidation tant que tous les GPA étudiants ne sont pas renseignés et que les vœux ne sont pas importés, afin de garantir que l'algorithme d'affectation dispose de données complètes et fiables.	M	3	S5
US-25	Authentification SSO	En tant qu' utilisateur , je veux m'authentifier via le SSO institutionnel (CAS/OIDC) afin d'accéder à mon espace personnalisé (étudiant ou administrateur) sans créer de compte séparé.	M	8	S5
US-26	Tableaux de bord analytiques et exports CTI	En tant qu' administrateur RI , je veux disposer de tableaux de bord analytiques et d'exports CTI (taux d'affectation par département, évolution des flux entrants/sortants, destinations les plus demandées, indicateurs CTI) afin d'alimenter le pilotage opérationnel et la réflexion stratégique sur les partenariats.	M	8	S5

6.2 Planification des sprints

Le développement MVP est découpé en cinq sprints de deux à trois semaines chacun, suivant une progression de l'infrastructure vers les fonctionnalités visibles. Les sprints sont ordonnés par dépendances techniques : rien dans le Sprint 2 ne peut exister sans le Sprint 1, et l'algorithme du Sprint 3 dépend directement des données produites par le Sprint 2. Les fonctionnalités classées *Could Have* restent hors engagement initial et ne seront intégrées qu'après validation complète du MVP, selon la capacité disponible. Le tableau 6.3 détaille la répartition.

TABLE 6.3 – Planification des sprints

Sprint	Thème	User stories couvertes
Sprint 1	Fondations techniques	US-01 à US-03 — Référentiels de base (sync MoveON), année universitaire et FSM, accords et quotas (sync MoveON)
Sprint 2	Données et traçabilité	US-04 à US-07 — Audit log, données étudiants (import Pégase), vœux étudiants (import MoveON), rapport d'import
Sprint 3	Algorithme d'affectation	US-08 à US-10 — Algorithme Gale-Shapley, supervision et publication des résultats, consultation des résultats par l'étudiant
Sprint 4	Sources complémentaires et mobilités	US-11 à US-16 — Répartition des quotas par département (Hamilton), stages internationaux (import Eudonet), mobilités entrantes (import Excel), réconciliation sans INE, déclaration et validation des mobilités complémentaires
Sprint 5	CTI, espace étudiant et supervision	US-17 à US-27 — Calcul des durées CTI, indicateur étudiant, export CTI, catalogue des destinations, suivi de candidature, gestion mobilités entrantes, console de supervision, contrôle des prérequis FSM, SSO, tableaux de bord analytiques et exports CTI

6.2.1 Diagramme de Gantt

La figure 6.4 présente le calendrier de réalisation sur l'ensemble des sprints, en indiquant pour chaque tâche la semaine de début, la durée estimée et l'éventuel chevauchement avec les tâches adjacentes.

TABLE 6.4 – Diagramme de Gantt

Activité	Mois 1				Mois 2				Mois 3				Mois 4				Mois 5				Mois 6			
	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	S16	S17	S18	S19	S20	S21	S22	S23	S24
Phase 1 — Cadrage et analyse																								
Analyse des besoins																								
Rédaction CDC / spécifications																								
Modélisation des données																								
Conception architecture																								
Phase 2 — Environnement et DevOps																								
Setup Docker / environnement																								
Pipeline CI/CD (Git-Lab/GitHub)																								
Configuration PostgreSQL																								
Setup Django-Q2 / MinIO																								
Monitoring (Sentry / Grafana)																								
Phase 3 — Développement itératif																								
Sprint 1																								
Sprint 2																								
Sprint 3																								
Sprint 4																								

Activité	Mois 1				Mois 2				Mois 3				Mois 4				Mois 5				Mois 6			
	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	S16	S17	S18	S19	S20	S21	S22	S23	S24
Sprint 5																								
Phase 4 — Tests et qualité																								
Tests unitaires (continus)																								
Tests d'intégration (continus)																								
Tests de charge (Locust)																								
Audit sécurité (OWASP ZAP)																								
UAT avec service RI																								
Phase 5 — Livraison																								
Documentation technique																								
Déploiement staging																								
Livraison finale																								

6.3 Architecture logicielle

6.3.1 Vue d'ensemble

La plateforme repose sur une architecture en cinq couches qui s'appuie sur le modèle 3-tiers classique tout en l'enrichissant d'une couche asynchrone dédiée aux traitements longs. Chaque couche a une frontière bien définie : aucun composant Next.js n'accède à la base de données directement, aucune tâche asynchrone ne peut bloquer une requête HTTP.

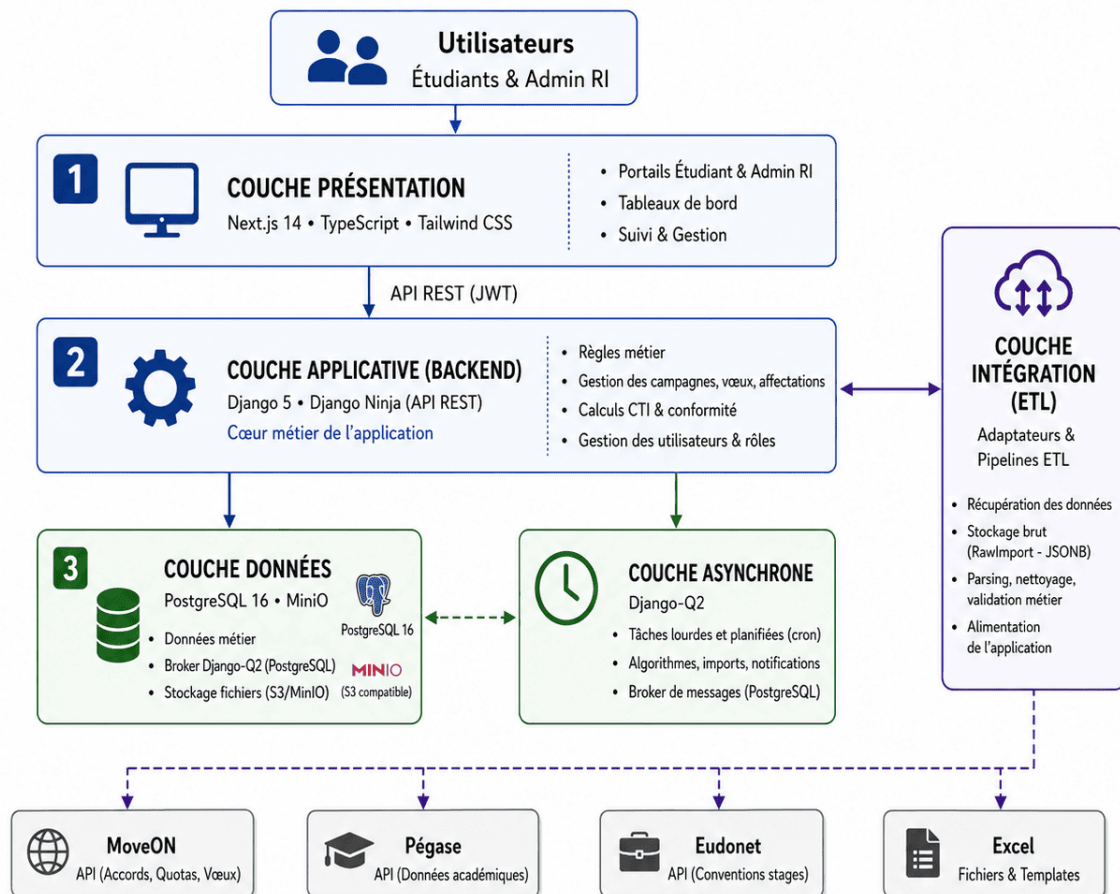


FIGURE 6.1 – Architecture logicielle

La figure 6.1 donne une vue d'ensemble du système. Les flux sont unidirectionnels et bien délimités. Le frontend Next.js 14 s'adresse exclusivement à l'API Django Ninja via des appels REST sur HTTPS — jamais directement à la base de données. Django-Q2 s'exécute dans un processus séparé et partage le même PostgreSQL que l'applicatif, ce qui lui permet de fonctionner comme broker de messages sans service externe supplémentaire. PostgreSQL et MinIO sont des services internes non accessibles depuis l'extérieur du réseau Docker. Les adaptateurs ETL sont physiquement séparés des modules métier, précisément parce qu'ils dépendent de sources externes (MoveON, Pégase, Eudonet) dont la disponibilité n'est pas garantie. L'authentification passe par le CAS OIDC de Toulouse INP : à chaque requête, le token JWT est validé et le rôle (ADMIN_RI ou ETUDIANT) est extrait pour contrôler les accès.

6.3.2 Couche présentation — Next.js 14

La couche présentation est développée en **Next.js 14** avec **TypeScript** et **Tailwind CSS**. Elle expose deux espaces fonctionnellement distincts, séparés dès le middleware de routage.

Espace administrateur RI : pilotage de campagne, gestion des quotas par accord et par département, supervision des affectations avant publication, instruction des demandes de mobilités complémentaires, tableaux de bord de conformité CTI.

Espace étudiant : consultation du catalogue de destinations, suivi de l'état de candidature, déclaration d'expériences complémentaires, indicateur personnel de durée CTI.

Toutes les communications entre le frontend et le backend passent par l'API REST, en JSON sur HTTPS, avec un token JWT joint à chaque requête.

6.3.3 Couche applicatif — Django 5 + Django Ninja

Le backend concentre la logique métier du système. Il est découpé en modules Django autonomes, chacun responsable d'un domaine précis, selon les principes DDD décrits au chapitre précédent. Cette organisation garantit qu'une modification dans le domaine de l'affectation ne peut pas affecter le comportement du domaine CTI ou des imports.

Django Ninja est utilisé pour exposer l'API REST. Chaque module définit ses propres schémas pour valider les données en entrée et structurer les réponses. L'authentification repose sur le middleware OIDC (CAS Toulouse INP), qui vérifie le token et associe l'utilisateur à chaque requête.

6.3.4 Couche traitement asynchrone — Django-Q2

Plutôt que de bloquer l'interface le temps qu'un calcul se termine, les traitements longs sont soumis à une file de tâches gérée par **Django-Q2**. Ce dernier utilise PostgreSQL comme broker, ce qui évite d'introduire Redis dans l'infrastructure. Deux familles de tâches y transitent :

- **Tâches déclenchées par événement** : lancement de l'algorithme d'affectation, recalcul des durées CTI après chaque transition FSM, génération des rapports d'import.
- **Tâches planifiées (cron)** : synchronisation nocturne avec MoveON, Pégase et Eudonet ; purge des données brutes (**RawImport**) traitées depuis plus de 90 jours.

6.3.5 Couche données — PostgreSQL 16

Un seul service de données est utilisé dans l'infrastructure : **PostgreSQL 16**. Ce choix simplifie les sauvegardes et la supervision. PostgreSQL remplit ici trois fonctions différentes :

- **Base de données relationnelle** : toutes les entités métier y sont stockées, avec des index composites sur `academic_year_id` pour les performances de filtrage par campagne.
- **Broker de messages Django-Q2** : la table `django_q_task` stocke les tâches en attente.
- **Stockage des données brutes (JSONB)** : la table `RawImport` conserve les réponses JSON brutes de chaque appel API avant transformation, garantissant la possibilité de relancer les imports.

Le stockage des fichiers (justificatifs de mobilités complémentaires, templates Excel) est assuré par **MinIO**, un serveur de stockage objet compatible S3, déployé comme container Docker interne.

6.3.6 Couche intégration — Adaptateurs ETL

Chaque source externe est encapsulée dans un adaptateur dédié, isolé du reste du code métier. Quatre adaptateurs ont été développés :

- `adapters/moveon.py` : extraction des accords bilatéraux, des quotas INP et des candidatures/vœux ;
- `adapters/pegase.py` : extraction des données académiques étudiants (INE, GPA trimestriel, département d'inscription) ;
- `adapters/eudonet.py` : extraction des conventions de stages à l'international ;
- `adapters/excel.py` : traitement des fichiers Excel standardisés (mobilités entrantes, données sans API disponible).

Tous les adaptateurs suivent le même enchaînement : récupération des données depuis la source → stockage brut en JSONB (`RawImport`) → parsing et standardisation → validation métier → persistance en base → génération du rapport d'import. En cas d'indisponibilité de la source, un mécanisme de retry exponentiel (trois tentatives) est appliqué avant qu'une alerte soit émise via **Sentry**.

6.3.6.1 Stratégie d'upsert

À la phase de persistance, chaque enregistrement est traité selon une logique d'upsert. La clé métier (généralement l'INE ou l'identifiant de l'accord) permet de déterminer si l'enregistrement est nouveau ou déjà connu. S'il est nouveau, il est inséré. S'il existe, ses champs sont comparés un à un avec la version en base : en l'absence de différence, l'enregistrement est ignoré ; si un champ a changé (par exemple un GPA révisé dans Pégase ou une date de fin de stage corrigée dans Eudonet), la mise à jour est appliquée et l'écart est consigné dans le journal d'audit. Cela permet au service RI de savoir exactement ce qui a changé d'une synchronisation à l'autre.

6.3.6.2 Clé de jointure : l'INE

L'identifiant national étudiant (INE) est la clé partagée entre MoveON, Pégase, Eudonet et le CAS institutionnel. Toutes les données importées sont croisées via l'INE pour construire une vue unifiée du dossier étudiant.

Lorsque l'INE est absent d'une source (notamment Eudonet), une stratégie de réconciliation est appliquée : matching déterministe sur le triplet nom/prénom/date de naissance, avec détection des cas ambigus remontés pour validation manuelle par le service RI.

6.3.6.3 Stratégie de synchronisation

Le tableau 6.5 récapitule, pour chaque source externe, le déclencheur de synchronisation et les particularités de traitement associées.

Source	Déclencheur	Remarque
MoveON	Tâche Django-Q2 planifiée (phases actives) + manuelle avant lancement de l'algorithme	Lecture seule — aucune écriture vers MoveON
Pégase	Initialisation de campagne + à la demande	GPA figé à la <code>gpa_freeze_date</code> définie par l'admin
Eudonet	Périodique (fin de période de stage) + à la demande	Réconciliation INE si absent
Excel	Upload manuel par l'administrateur	Fallback pour toutes les sources en cas d'indisponibilité API

TABLE 6.5 – Stratégie de synchronisation par source

6.3.7 Sécurité et contrôle d'accès

6.3.7.1 Flux d'authentification OIDC

L'application délègue entièrement l'authentification au CAS institutionnel de Toulouse INP via OIDC. Aucun mot de passe n'est stocké côté applicatif. Le flux se déroule en quatre étapes :

1. L'utilisateur accède à l'application et est redirigé vers le portail SSO institutionnel.
2. Après authentification CAS, un token OIDC (JWT) est émis et transmis à l'application.
3. Le backend décode le token, extrait l'INE et le groupe LDAP de l'utilisateur, et détermine automatiquement son rôle (`ADMIN_RI` ou `ETUDIANT`).
4. Le frontend reçoit le rôle et affiche l'espace correspondant. Les tokens ont une durée de vie courte (15 minutes) et sont renouvelés silencieusement sans interrompre la navigation.

6.3.7.2 Contrôle d'accès (RBAC)

Les rôles sont déduits automatiquement depuis l'annuaire LDAP institutionnel, sans gestion manuelle dans l'application :

- **ADMIN_RI** : accès complet aux fonctions d'administration — imports, lancement de l'algorithme, validations, exports CTI. Toute action est enregistrée dans le journal d'audit.
- **ETUDIANT** : accès strictement limité à son propre dossier. Un contrôle au niveau de l'objet (*object-level permission*) est appliqué sur chaque endpoint exposant des données personnelles — un étudiant ne peut pas accéder au dossier d'un autre.

6.3.7.3 Gestion du renouvellement des tokens JWT

Les tokens JWT ont une durée de vie de 15 minutes pour limiter l'exposition en cas d'interception. Côté Next.js, NextAuth gère le renouvellement silencieusement : quand le token approche de son expiration, un flux de rafraîchissement est initié via le endpoint OIDC du CAS sans interrompre la session. Côté Django, chaque requête est vérifiée par un middleware qui contrôle la signature et l'expiration du token ; un token expiré retourne un 401 et force le frontend à initier un nouveau cycle. Un token révoqué au niveau du CAS ne peut donc pas être utilisé pour accéder au backend, même si le frontend n'a pas encore détecté l'invalidation.

6.3.7.4 Journal d'audit

La traçabilité des actions administratives est assurée par `django-auditlog`, sans code supplémentaire dans les vues. Les événements enregistrés couvrent :

- Authentifications réussies et tentatives échouées ;
- Toutes les transitions FSM (changements d'état d'une année universitaire, d'un dossier, d'une affectation) ;
- Modifications manuelles des affectations par le service RI (avec champ *motif* obligatoire) ;
- Lancements d'imports ETL et résultats (source, nombre d'enregistrements traités, erreurs) ;
- Validations ou rejets de mobilités complémentaires ;
- Exports de rapports CTI.

Chaque entrée contient l'identifiant de l'auteur, l'horodatage UTC, l'entité concernée (table et clé primaire), l'action effectuée, et les valeurs avant/après modification. Le journal est stocké dans une table PostgreSQL dédiée (`auditlog_logentry`), accessible en lecture seule. La politique de rétention est de sept ans. Aucune entrée ne peut être modifiée ou

supprimée depuis l’application — toute suppression nécessite un accès direct à la base, lui-même tracé au niveau opérateur.

6.3.7.5 Protection des données (RGPD)

La plateforme traite des données personnelles étudiantes (INE, email, date de naissance, résultats académiques) soumises au RGPD. Quatre mesures techniques ont été intégrées dès la conception :

- **Chiffrement au repos** : les champs sensibles sont chiffrés en base via `pgcrypto` ; leur lecture requiert la clé de déchiffrement qui n’est jamais stockée dans le code.
- **Chiffrement en transit** : TLS est obligatoire sur tous les endpoints ; les communications HTTP sont systématiquement redirigées vers HTTPS par Nginx.
- **Stockage des justificatifs** : les fichiers uploadés (justificatifs de mobilités complémentaires) sont chiffrés côté serveur dans MinIO (SSE-S3) et supprimés automatiquement après validation et expiration du délai de conservation.
- **Droit à l’oubli** : une procédure d’anonymisation est prévue après diplôme — les données nominatives sont effacées, les statistiques agrégées sont conservées pour les rapports CTI.

6.4 Architecture physique et déploiement

6.4.1 Vue d’ensemble

L’application s’exécute sur un serveur on-premise hébergé par Toulouse INP. Tous les services sont conteneurisés via **Docker Compose**, ce qui garantit que l’environnement de développement et l’environnement de production sont structurellement identiques — éliminer les surprises du déploiement était une contrainte dès le départ.

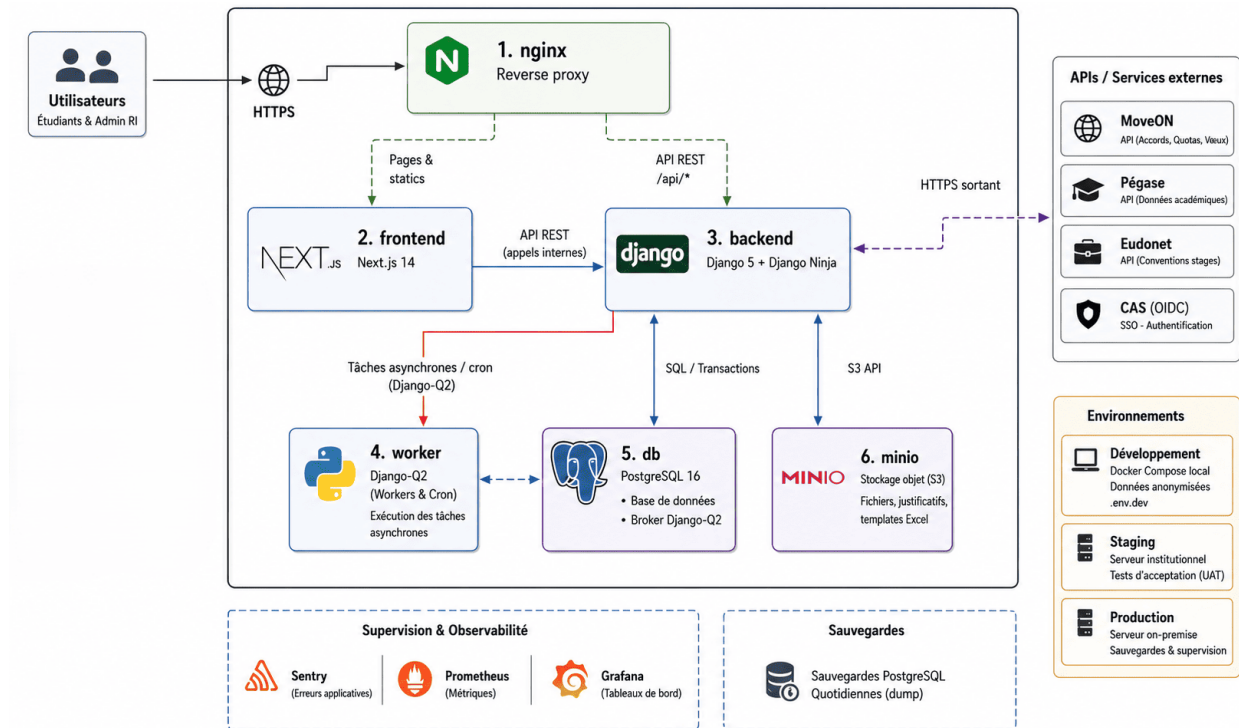


FIGURE 6.2 – Architecture physique

La figure 6.2 détaille les flux réseau. Tout le trafic entrant passe par **Nginx**, le seul container exposé à l'extérieur (ports 80 et 443). Nginx redirige le HTTP vers HTTPS, sert les fichiers statiques Next.js et route les appels API vers **backend:8000**. PostgreSQL, MinIO et le worker Django-Q2 sont confinés dans le réseau Docker interne — aucune connexion directe depuis l'extérieur n'est possible. Les appels vers les APIs institutionnelles (MoveON, Pégase, Eudonet, CAS) partent exclusivement du container **backend** en HTTPS sortant. Sentry est externe à l'infrastructure et reçoit les rapports d'erreur du backend et du frontend. Prometheus (collecte de métriques) et Grafana (visualisation) tournent en interne, accessibles uniquement depuis le réseau institutionnel.

6.4.2 Containers Docker

Six containers composent l'infrastructure. Le tableau 6.6 en donne le détail.

Container	Image	Rôle
nginx	nginx:alpine	Reverse proxy : SSL/TLS, fichiers statiques, routage vers Django et Next.js
frontend	node:20-alpine	Application Next.js (SSR + API routes)
backend	python:3.12-slim	Django 5 + Django Ninja
worker	python:3.12-slim	Django-Q2 cluster (tâches asynchrones et cron)
db	postgres:16-alpine	PostgreSQL — base de données et broker Django-Q2
minio	minio/minio	Stockage objet S3 (justificatifs, fichiers Excel)

TABLE 6.6 – Containers Docker et leurs rôles

6.4.3 Flux réseau

Nginx est le seul point d'entrée du système : les requêtes vers `/api/*` sont acheminées vers `backend:8000`, toutes les autres vers `frontend:3000`. Le container `backend` dialogue avec la base de données pour les opérations métier et avec MinIO pour la gestion des fichiers. Le `worker` Django-Q2 utilise le même PostgreSQL comme file de messages — aucun Redis ni RabbitMQ n'est nécessaire. Les appels sortants vers MoveON, Pégase, Eudonet et le CAS passent uniquement par le container `backend`.

6.4.4 Environnements

Trois environnements coexistent, avec une parité de configuration maximale entre eux :

- **Développement** : Docker Compose local, données anonymisées, variables d'environnement dans `.env`. Chaque développeur dispose d'un environnement reproductible en une commande.
- **Staging** : copie conforme de la production hébergée sur le serveur institutionnel, utilisée pour les tests d'acceptation (UAT) avec le service RI avant chaque mise en production.
- **Production** : déploiement final sur le serveur on-premise, avec sauvegardes PostgreSQL quotidiennes, alertes via Sentry et métriques via Prometheus/Grafana.

6.4.5 CI/CD

Le pipeline CI/CD sur **GitHub Actions** s'exécute à chaque push. Trois étapes s'enchaînent :

- **Lint et tests** : le linter `ruff` vérifie la qualité du code Python, puis `pytest` exécute la suite de tests complète.
- **Build** : construction des images Docker et vérification que l'ensemble des services démarre sans erreur.
- **Déploiement staging** : déploiement automatique sur l'environnement de staging à chaque merge sur `main`, sans intervention manuelle.

6.4.6 Stratégie de migration de schéma

Les migrations Django sont appliquées automatiquement avant le démarrage du serveur applicatif, ce qui garantit que le code et le schéma de base de données sont toujours cohérents. La stratégie retenue est celle des migrations additives en deux temps : on ajoute d'abord le nouveau champ (migration non destructive), et on supprime les éléments devenus inutiles dans une migration ultérieure, une fois que le code de production n'y fait plus référence. Cette approche permet de déployer sans fenêtre de maintenance et sans risque de rupture entre le code et le schéma. Chaque fichier de migration est versionné dans le dépôt git au même titre que le code, et relu avant intégration.

6.4.7 Stratégie de rollback

Si un incident survient après un déploiement, le rollback se déroule en deux temps. L'image Docker précédente est remise en service en ciblant son tag de version. Si la migration de schéma était irréversible (suppression de colonne par exemple), une restauration depuis la sauvegarde pré-déploiement est effectuée — c'est pourquoi une sauvegarde complète est systématiquement déclenchée juste avant tout déploiement en production. Pour les migrations réversibles, un retour en arrière via la commande Django est possible sans restauration complète.

6.4.8 Politique de sauvegarde

Deux niveaux de sauvegarde sont mis en place. Des sauvegardes logiques PostgreSQL sont générées chaque nuit et conservées 30 jours sur un volume distinct du volume de données. Des sauvegardes physiques hebdomadaires couvrent l'intégralité des données et sont archivées pendant six mois, permettant une restauration fine en combinaison avec les journaux transactionnels.

Un test de restauration est effectué chaque mois sur l'environnement staging : on restaure la sauvegarde la plus récente et on vérifie l'intégrité des données via un script de contrôle (comptages par table, contraintes référentielles). Cette procédure mesure le *Recovery Time Objective* réel et permet de détecter un problème avant qu'il ne se révèle en production.

6.4.9 Gestion des secrets

Aucun secret n’est présent dans le dépôt git. Les variables sensibles — `SECRET_KEY`, mots de passe PostgreSQL, clés d’API MoveON, Pégase, Eudonet, tokens MinIO — sont injectées au runtime via un fichier `.env.prod` stocké uniquement sur le serveur de production. Le `.gitignore` exclut tous les fichiers `.env.*`, à l’exception d’un fichier de référence avec des valeurs fictives fourni pour le développement local.

6.5 Modèle de données

Le modèle de données traduit en structures relationnelles les entités métier identifiées dans l’analyse. Il est organisé en huit packages thématiques, chacun correspondant à un domaine métier distinct. Cette section en présente la logique d’ensemble ; les diagrammes de classes détaillés de chaque sprint sont présentés dans le chapitre de réalisation.

6.5.1 Organisation par packages

Le modèle s’articule autour de huit packages dont les responsabilités sont résumées dans le tableau suivant.

Package	Responsabilité
Reference	Référentiels stables partagés par l’ensemble du système : pays, régions CTI, départements N7.
Academic Year	Axe temporel central. Chaque campagne de mobilité est rattachée à une année universitaire dont le cycle de vie est piloté par une machine à états.
Students	Données académiques des étudiants importées depuis Pégase : <code>Student</code> , inscriptions annuelles (<code>AnnualEnrollment</code>), parcours pédagogique (<code>Parcours</code>), relevés de GPA gelés à date fixe.
Institutions	Universités partenaires (synchronisées depuis MoveON) et organismes d’accueil pour les stages.
Agreements & Quotas	Accords de mobilité (<code>Agreement</code>) et leur hiérarchie de quotas : déclinaison annuelle par accord (<code>AgreementYear</code>), liste des départements autorisés par accord (<code>AgreementDepartment</code>), et répartition annuelle du quota N7 par département (<code>AgreementYearDepartment</code>).
Applications	Dossiers de candidature, vœux ordonnés et affectations produites par l’algorithme ou saisies manuellement.
Mobilities	Quatre types de mobilités : sortantes académiques, stages internationaux (Eudonet), complémentaires déclarées par l’étudiant, et entrantes (import Excel). La durée CTI consolidée est calculée automatiquement.
Import	Stockage brut (JSONB) des données reçues de chaque source avant transformation, garantissant la reprise sur erreur et la traçabilité des imports.

TABLE 6.7 – Packages du modèle de données

6.5.2 Principes structurants

Trois décisions de conception traversent l'ensemble du modèle et méritent d'être explicitées.

6.5.2.1 L'année universitaire comme axe temporel

Toutes les entités opérationnelles — dossiers, vœux, affectations, mobilités, imports — sont rattachées à une `AcademicYear`. Cette clé de partitionnement logique garantit deux propriétés : l'historique des campagnes passées est complet et consultable, et dès qu'une année est clôturée, ses données basculent automatiquement en lecture seule.

6.5.2.2 L'INE comme identifiant pivot

L'identifiant national étudiant (INE) est le seul identifiant commun à toutes les sources — MoveON, Pégase, Eudonet et le CAS OIDC. Tous les imports sont croisés via l'INE pour construire une vue unifiée du dossier étudiant. Quand un enregistrement arrive sans INE valide (cas courant dans Eudonet), une réconciliation déterministe sur le triplet nom / prénom / date de naissance est appliquée, et les cas ambigus sont remontés pour validation manuelle par le service RI.

6.5.2.3 Hiérarchie quotas INP / quotas N7

MoveON ne connaît les quotas qu'à l'échelle de l'INP. Pour gérer la répartition interne à l'ENSEEIH, le modèle introduit quatre niveaux : le quota INP importé depuis MoveON, l'estimation N7 calculée par l'administrateur, un éventuel ajustement manuel, et la répartition finale par département pédagogique. Ce découpage assure à la fois la conformité avec MoveON et la granularité nécessaire à l'algorithme d'affectation.

6.6 Machines à états finis (FSM)

6.6.1 Principe et implémentation

Dans un système comme celui-ci, certaines entités ne peuvent pas changer d'état librement. Une campagne clôturée ne doit pas pouvoir être rouverte, une affectation validée ne doit pas être modifiable sans trace, et l'algorithme ne doit pas s'exécuter sur des données incomplètes. Ces contraintes sont encodées dans des machines à états finis, implémentées via `django-fsm`. Le champ de statut est déclaré avec l'option `protected=True` : toute tentative de modification directe du statut — sans passer par une méthode de transition nommée — lève immédiatement une exception. Il n'est donc pas possible de contourner ces règles, même par une opération en base de données effectuée manuellement.

6.6.2 FSM de l'année universitaire

C'est la machine à états centrale du système. Elle gouverne l'accès à toutes les fonctionnalités de la campagne. La figure 6.3 en présente les états et transitions.

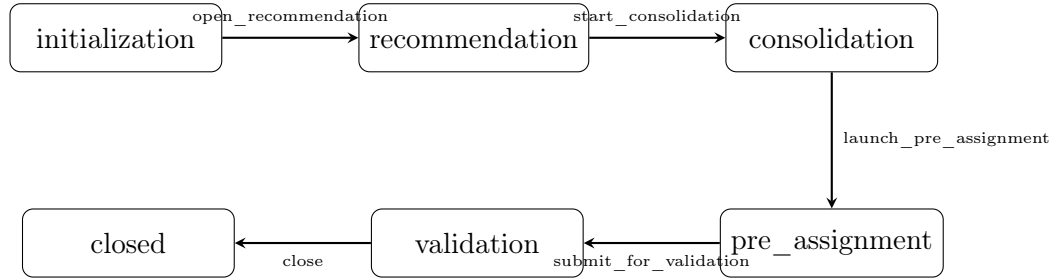


FIGURE 6.3 – Machine à états — AcademicYear

Chaque transition marque un cap dans le déroulement de la campagne.

- **initialization** → **recommendation** (`open_recommendation`) : les étudiants peuvent désormais consulter le catalogue de destinations et formuler leurs vœux.
- **recommendation** → **consolidation** (`start_consolidation`) : la fenêtre de saisie des vœux se ferme, le GPA est gelé à la date du jour, et la synchronisation finale avec MoveON est déclenchée.
- **consolidation** → **pre_assignment** (`launch_pre_assignment`) : l'algorithme Gale-Shapley est soumis à Django-Q2 pour exécution asynchrone.
- **pre_assignment** → **validation** (`submit_for_validation`) : les propositions d'affectation sont soumises à l'examen du service RI, qui peut les ajuster avant publication.
- **validation** → **closed** (`close`) : les résultats sont publiés et toutes les données de la campagne passent en lecture seule.

Chaque transition est réservée aux utilisateurs `ADMIN_RI`. Le tableau ci-dessous liste les conditions que le système vérifie avant d'autoriser chaque passage d'état, ainsi que le comportement en cas d'échec.

Transition	Préconditions (guard conditions)	Comportement en cas d'échec
open_recommendation	Accords importés depuis MoveON, quotas par département configurés	Exception levée, message d'erreur précis renvoyé au frontend
start_consolidation	Au moins un vœu saisi par les étudiants, date limite de saisie atteinte ou dépassée	Exception levée si aucun vœu importé
launch_pre_assignment	GPA gelé (synchronisation Pégase terminée), vœux importés depuis MoveON	Exception levée, tâche Django-Q2 non déclenchée
submit_for_validation	Algorithme exécuté sans erreur, toutes les affectations en état proposed	Transition bloquée si des affectations sont en état intermédiaire
close	Validation manuelle explicite du service RI sur les propositions	Transition refusée tant que des affectations restent non validées

TABLE 6.8 – Conditions préalables (guard conditions) des transitions FSM

6.6.3 FSM des entités métier

La FSM de l'année universitaire n'est pas la seule : plusieurs entités métier disposent de leur propre cycle de vie, encodé de la même façon. Le tableau 6.9 en donne un aperçu synthétique.

Entité	États et transitions
ApplicationFile	open → submitted → processing → closed
StudentWish	candidate → selected → assigned rejected
Assignment	proposed → validated → published cancelled
OutgoingMobility	planned → ongoing → completed cancelled
ComplementaryMobility	submitted → under_review → validated rejected
IncomingMobility	nominated → imported → registered → active

TABLE 6.9 – FSM des entités métier

Chaque transition sur **OutgoingMobility**, **InternationalInternship** ou **ComplementaryMobility** déclenche automatiquement une tâche Django-Q2 de recalcul de **MobilityDuration** pour l'étudiant concerné, garantissant que l'indicateur CTI est toujours à jour.

6.7 Flux de données : de la source au workflow

Chaque information importée dans le système suit un même parcours balisé, du fichier brut jusqu'à son intégration dans les entités métier. Ce parcours est délibérément découpé

pour garantir qu’aucune donnée non validée ne peut déclencher un workflow, et que chaque étape est indépendamment traçable et rejouable.

1. **Récupération** : la tâche Django-Q2 interroge la source (API MoveON, Pégase, Eudonet ou fichier Excel). La réponse brute est immédiatement sérialisée en JSONB dans `RawImport`. Aucune transformation n’a encore eu lieu à ce stade — ce qui signifie que si un traitement ultérieur échoue, l’import peut être rejoué sans rappeler la source.
2. **Nettoyage et validation** : un pipeline Python normalise les chaînes, croise les INE, détecte les doublons et vérifie les règles métier. Les enregistrements qui ne passent pas la validation sont consignés dans le rapport d’import avec le motif de rejet, sans interrompre le reste du traitement.
3. **Persistence** : les enregistrements validés sont intégrés en base par upsert atomique, rattachés à l’année universitaire en cours.
4. **Déclenchement des workflows** : chaque entité créée entre dans l’état initial de sa FSM. Un vœu importé commence en statut `candidate` ; une affectation créée par l’algorithme commence en `proposed`. Toute progression ultérieure passe obligatoirement par une transition FSM explicite.
5. **Supervision** : l’administrateur consulte le rapport d’import, traite les anomalies signalées et confirme le passage à la phase suivante de la campagne.

La figure 6.4 illustre ces cinq étapes successives.

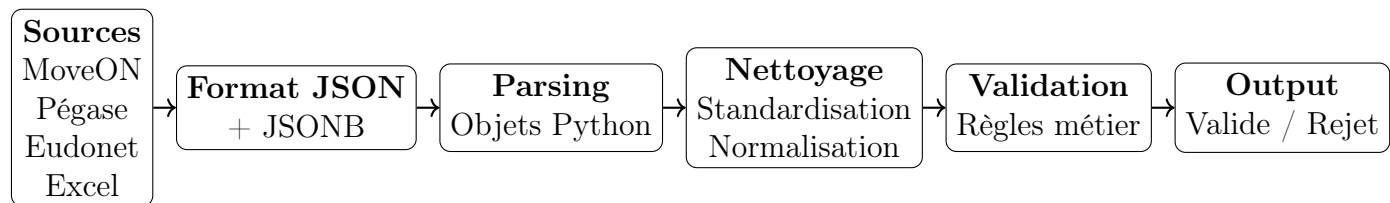


FIGURE 6.4 – Pipeline global de traitement des données

6.7.1 Détail de l’étape de nettoyage et validation

L’étape de nettoyage et validation est la plus délicate du pipeline : c’est là que les données brutes issues de sources hétérogènes deviennent exploitables par le système métier. Elle s’intercale entre la phase de récupération (stockage JSONB) et la persistance en base.

6.7.1.1 Description du processus de transformation

Les données importées arrivent sous des formats incompatibles : une API REST renvoie du JSON, Eudonet exporte du XML, les fichiers Excel du service RI utilisent des colonnes nommées différemment selon l’agent qui les a créés. Le pipeline de transformation commence donc par convertir toutes ces entrées dans une représentation intermédiaire commune, avant d’appliquer les mêmes règles de nettoyage quel que soit l’origine. Cette approche modulaire permet d’ajouter une nouvelle source sans modifier la logique de validation.

6.7.1.1.1 Standardisation des formats

- Normalisation des chaînes de caractères : suppression des espaces en début/fin, homogénéisation de la casse pour les codes (INE, codes ISO pays).
- **Gestion des accents :**
 - Les accents sont conservés pour les champs descriptifs (noms propres, universités, pays) afin de préserver l'intégrité sémantique des données.
 - Lors des opérations de recherche, de comparaison ou de détection de similarité (*matching*), les comparaisons sont effectuées de manière insensible aux accents afin d'éviter les divergences liées aux variations d'écriture.
- Uniformisation des formats de dates (ISO 8601).
- Conversion des types (ex : chaînes vers entiers pour les identifiants).

6.7.1.1.2 Normalisation sémantique

- Harmonisation des noms d'universités via une table de correspondance (*lookup table*) :
 - Les données sources peuvent contenir différentes variantes pour une même université (abréviations, différences de langue, fautes de frappe).
 - Une table de correspondance permet d'associer ces variantes à une valeur unique de référence.
 - Cette approche garantit la cohérence des données et évite les duplications logiques lors des traitements.
- Normalisation des pays selon un référentiel standard (ISO 3166).
- Mapping des valeurs hétérogènes vers des valeurs métier uniques (ex : "FR", "France", "france" → "France").

6.7.1.1.3 Gestion des valeurs manquantes

- Identification des champs obligatoires (INE, université, type de mobilité).
- Si une valeur obligatoire est manquante, l'enregistrement est rejeté et consigné dans le rapport d'import avec la raison de l'erreur.
- Si les champs manquants ne sont pas critiques, l'enregistrement est conservé et peut être sauvegardé en base de données.

6.7.1.1.4 Détection et gestion des doublons

- Les doublons sont identifiés à partir d’une clé composite construite à partir de plusieurs attributs pertinents (par exemple : INE, entité concernée, période), permettant de représenter de manière unique un enregistrement métier.
- La composition de cette clé peut varier selon le type de donnée traité (étudiant, souhait, accord, etc.), afin de s’adapter aux spécificités de chaque entité.
- Chaque enregistrement est comparé aux données existantes afin de détecter les occurrences identiques.
- En cas de doublon exact, l’enregistrement est ignoré afin d’éviter la duplication des données.
- Les cas détectés sont tracés dans le rapport d’import pour assurer la traçabilité.

6.7.1.1.5 Validation métier

- Vérification de la cohérence avec les données Pégase (existence de l’étudiant via INE).
- Contrôle des règles métier (nombre maximum de vœux, compatibilité des destinations).

6.7.1.1.6 Traçabilité et gestion des erreurs

- Journalisation des erreurs dans un rapport d’import structuré.
- Stockage des données rejetées avec leur motif (audit et reprise).
- Attribution d’un identifiant de traitement pour chaque batch.

6.7.1.2 Architecture du pipeline de nettoyage

Le pipeline de nettoyage est conçu comme un module autonome intégré dans le flux global. Il fonctionne par batch afin d’assurer l’isolation des traitements et la robustesse du système.

- **Entrée** : données JSON issues du pipeline global.
- **Parsing** : transformation en objets manipulables.
- **Nettoyage** : standardisation et normalisation.
- **Validation** : application des règles métier.
- **Sortie** :
 - Données valides → persistance.
 - Données invalides → rapport d’erreurs.

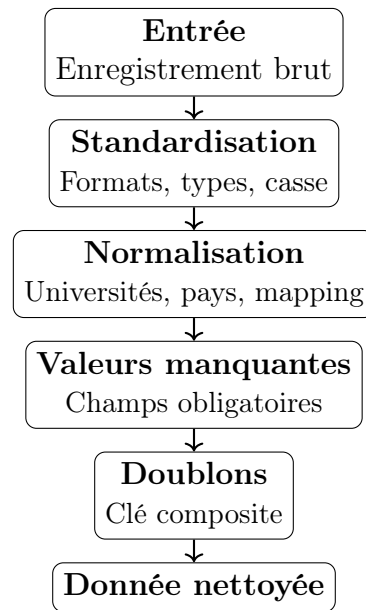


FIGURE 6.5 – Pipeline détaillé de l’étape de nettoyage des données

Cette architecture garantit :

- une **traçabilité complète** grâce au stockage JSONB,
- une **robustesse** via le traitement isolé de chaque enregistrement,
- une **reprise sur erreur** par batch,
- une **maintenabilité** grâce à la modularité du pipeline.

6.7.1.3 Technologies et outils utilisés

- **Python** : traitement et transformation des données.
- **Pandas** : manipulation et nettoyage des datasets.
- **Regex** : validation et transformation des formats.
- **PostgreSQL (JSONB)** : stockage temporaire des données brutes.

Ce pipeline garantit qu’aucune donnée non validée n’atteint la couche métier. Chaque rejet est documenté dans le rapport d’import avec le motif exact, ce qui permet à l’administrateur de corriger la source ou de saisir manuellement les données manquantes.

6.8 Gestion des risques

Concevoir un système qui s’interface avec plusieurs sources institutionnelles implique d’accepter dès le départ que certaines choses ne se passeront pas comme prévu. Le tableau ci-dessous liste les risques les plus significatifs identifiés en phase de conception et les mesures retenues pour en limiter l’impact.

TABLE 6.10 – Matrice des risques

Risque	Prob.	Impact	Mitigation
INE absent dans une source externe (Eudonet, MoveON)	Haute	Élevé	Réconciliation par matching nom / prénom / date de naissance. Cas ambigus remontés pour validation manuelle par le service RI.
API externe indisponible pendant une campagne	Moyenne	Élevé	Import Excel de secours documenté pour chaque source. Cache local des données de la dernière synchronisation réussie.
Dérive du périmètre en cours de PFE	Haute	Moyen	Priorisation stricte MVP / V2. Les fonctionnalités V2 (chatbot, recommandation) restent hors engagement initial et ne sont activées que dans une réserve conditionnelle, après validation du MVP et selon la capacité disponible.
Résistance au changement du service RI	Moyenne	Moyen	Implication du service RI dans les tests d'acceptation (UAT). Interface proche des pratiques Excel actuelles.
Fuite de données personnelles	Faible	Très élevé	Chiffrement des données sensibles (pgcrypto), cloisonnement strict par rôle (RBAC), audit log de toutes les actions.
Démarrage à froid du moteur de recommandation	Haute	Faible	Les premières campagnes utiliseront des heuristiques simples (GPA + département). Le modèle s'affine progressivement à mesure que l'historique s'accumule (2-3 ans de données suffisent).

Risque	Prob.	Impact	Mitigation
Performance de l'algorithme d'affectation sous contraintes multiples	Faible	Moyen	Gale-Shapley est en $O(n^2)$: parfaitement adapté à quelques centaines d'étudiants. Un test de charge Locust sera réalisé avant la mise en production pour vérifier les temps de réponse.

Conclusion

La conception du système est désormais complète : le backlog structure le développement en cinq sprints MVP, l'architecture définit les responsabilités de chaque couche et container, le modèle de données formalise les entités métier, et les machines à états garantissent l'intégrité des cycles de vie. Le chapitre suivant présente la réalisation sprint par sprint.

Réalisation

Plan

Introduction	89
7.1 Sprint 1 — Socle technique et référentiels	89
7.1.1 Objectif du sprint	89
7.1.2 Sprint Backlog — Sprint 1	89
7.1.3 Conception détaillée	91
7.1.3.1 Diagramme de classes	91
7.1.4 Implémentation	92
7.1.5 Tests	92
7.2 Sprint 2 — Données et traçabilité	92
7.2.1 Objectif du sprint	92
7.2.2 Sprint Backlog — Sprint 2	93
7.2.3 Conception détaillée	95
7.2.3.1 Diagramme de classes	95
7.2.4 Implémentation	96
7.2.5 Tests	96
7.3 Sprint 3 — Algorithme d'affectation	96
7.3.1 Objectif du sprint	96
7.3.2 Sprint Backlog — Sprint 3	97
7.3.3 Conception détaillée	99
7.3.3.1 Diagramme de classes	99
7.3.4 Implémentation	99
7.3.5 Tests	99

Introduction

Ce chapitre couvre la phase de réalisation du projet, organisée en sprints successifs. Pour chaque sprint, le backlog, la conception détaillée, l’implémentation et les tests sont présentés conformément à la définition of done.

7.1 Sprint 1 — Socle technique et référentiels

Cette section présente la réalisation du Sprint 1, dédié à la mise en place du socle technique et des référentiels. Elle couvre successivement l’objectif du sprint, le backlog, la conception UML, l’implémentation et les tests.

7.1.1 Objectif du sprint

Le Sprint 1 a pour objectif d’implémenter les fondations irréversibles du système : référentiels stables (`Country`, `Department`), universités partenaires (`PartnerUniversity`), année universitaire avec sa machine à états à 6 phases (`AcademicYear`), accords et quotas (`Agreement`, `AgreementQuota`, `DepartmentQuota`) et stockage brut des imports (`RawImport`). Ces composants constituent la colonne vertébrale du système — toute fonctionnalité des sprints suivants en dépend directement.

Définition of Done. À l’issue de ce sprint, le système doit être exécutable via `docker compose up`, toutes les migrations Django doivent s’appliquer sans erreur, les tests unitaires doivent passer avec une couverture supérieure à 80 %, et le pipeline CI/CD doit être opérationnel sur GitHub Actions.

7.1.2 Sprint Backlog — Sprint 1

#	Tâche	Description	Critères d’acceptation	Prior.	SP
US-01 — Référentiels de base — <i>En tant qu’administrateur RI, je veux disposer d’un référentiel stable des pays, départements et universités partenaires afin que toutes les entités du système s’appuient sur des données de référence cohérentes.</i>					
01	Définir <code>Country</code> et <code>Department</code>	Créer les deux modèles de référentiel avec <code>iso2</code> unique, région CTI, <code>clean()</code> et <code>last_sync_moveon</code> .	Les modèles migrent sans erreur.	Haute	2
02	Définir <code>PartnerUniversity</code>	Créer le modèle avec <code>moveon_id</code> comme clé de jointure <code>MoveON</code> .	Le modèle migre sans erreur.	Haute	1

#	Tâche	Description	Critères d'acceptation	Prior.	SP
03	Page d'administration des référentiels	Créer une interface Next.js permettant à l'administrateur RI de consulter et filtrer les pays, départements et universités partenaires enregistrés.	La liste s'affiche, le filtre par nom fonctionne, les données sont chargées depuis l'API.	Haute	2
US-02 — Année universitaire et cycle de vie — <i>En tant qu'administrateur RI, je veux que le système gère le cycle de vie de l'année universitaire via une machine à états afin de contrôler chaque phase de la campagne de manière sécurisée.</i>					
04	Définir AcademicYear avec FSM	Définir le modèle AcademicYear en utilisant une machine à états finis (FSM), implémentée avec <code>django-fsm</code> , comprenant 6 états correspondant aux différentes phases de la campagne. Mettre en place des transitions sécurisées entre ces états, ainsi qu'une méthode <code>clean()</code> permettant de garantir la cohérence des dates.	Les transitions valides fonctionnent. Les invalides lèvent <code>TransitionNotAllowed</code> . <code>start_date > end_date</code> lève <code>ValidationError</code> .	Haute	2
05	Composant de gestion de l'année universitaire	Créer une interface Next.js affichant l'état courant de l'année universitaire avec les boutons de transition disponibles (selon la FSM) et les dates clés configurables par l'administrateur.	L'état affiché reflète la FSM. Les transitions non autorisées sont masquées. Les dates se sauvegardent via l'API.	Haute	2
US-03 — Accords et quotas — modèle de données — <i>En tant qu'administrateur RI, je veux disposer d'un modèle de données complet pour les accords et leurs quotas afin d'avoir une base fiable pour l'algorithme d'affectation.</i>					
06	Définir Agreement	Créer le modèle avec <code>moveon_id</code> , <code>code_moveon</code> unique, <code>agreement_type</code> , <code>valid_from/until</code> , <code>is_active</code> et <code>last_sync_moveon</code> .	Le modèle migre sans erreur. <code>is_valid_for_year()</code> retourne <code>False</code> pour un accord expiré ou inactif.	Haute	2
07	Définir AgreementQuota et DepartmentQuota	Créer les modèles de quotas avec hiérarchie INP → N7 estimé → N7 ajusté.	<code>get_effective_quota()</code> retourne l'ajusté si renseigné, sinon l'estimé. <code>is_eligible()</code> est fonctionnel.	Haute	2

#	Tâche	Description	Critères d'acceptation	Prior.	SP
08	Définir RawImport	Créer le modèle de stockage brut JSON avec <code>source</code> , <code>processed</code> , <code>error_message</code> et rattachement à <code>AcademicYear</code> .	Le modèle migre sans erreur. Les 4 sources sont disponibles. <code>processed=False</code> par défaut.	Haute	1
09	Page de consultation des accords et quotas	Créer une interface Next.js listant les accords par université partenaire, avec affichage des quotas INP, estimé N7 et ajusté N7 par département, et un indicateur visuel de validité de l'accord.	Les accords s'affichent avec leurs quotas effectifs. Les accords expirés ou inactifs sont signalés visuellement. L'ajustement manuel du quota N7 est possible depuis l'interface.	Haute	3

7.1.3 Conception détaillée

Cette section présente la conception UML détaillée des entités métier du Sprint 1.

7.1.3.1 Diagramme de classes

Le diagramme de la figure 7.1 modélise le domaine de gestion des accords de mobilité internationale de l'ENSEEIH. L'entité centrale est **Agreement**, qui représente un accord entre l'ENSEEIH et une université partenaire (**PartnerUniversity**), qualifiée par un cadre (**Category**) et des niveaux d'études (**Level**).

La structure des quotas est organisée sur deux niveaux : **AgreementYear** matérialise la déclinaison annuelle d'un accord avec le quota N7 global, et **AgreementYearDepartment** distribue ce quota par département. **AgreementDepartment** liste les départements autorisés par accord, indépendamment de l'année, et constitue la contrainte référentielle qui interdit d'allouer un quota à un département non autorisé.

AcademicYear pilote le cycle de vie de la campagne via une machine à états à six phases. Toutes les classes portent les champs d'audit `created_at`, `updated_at`, `created_by` et `updated_by`.

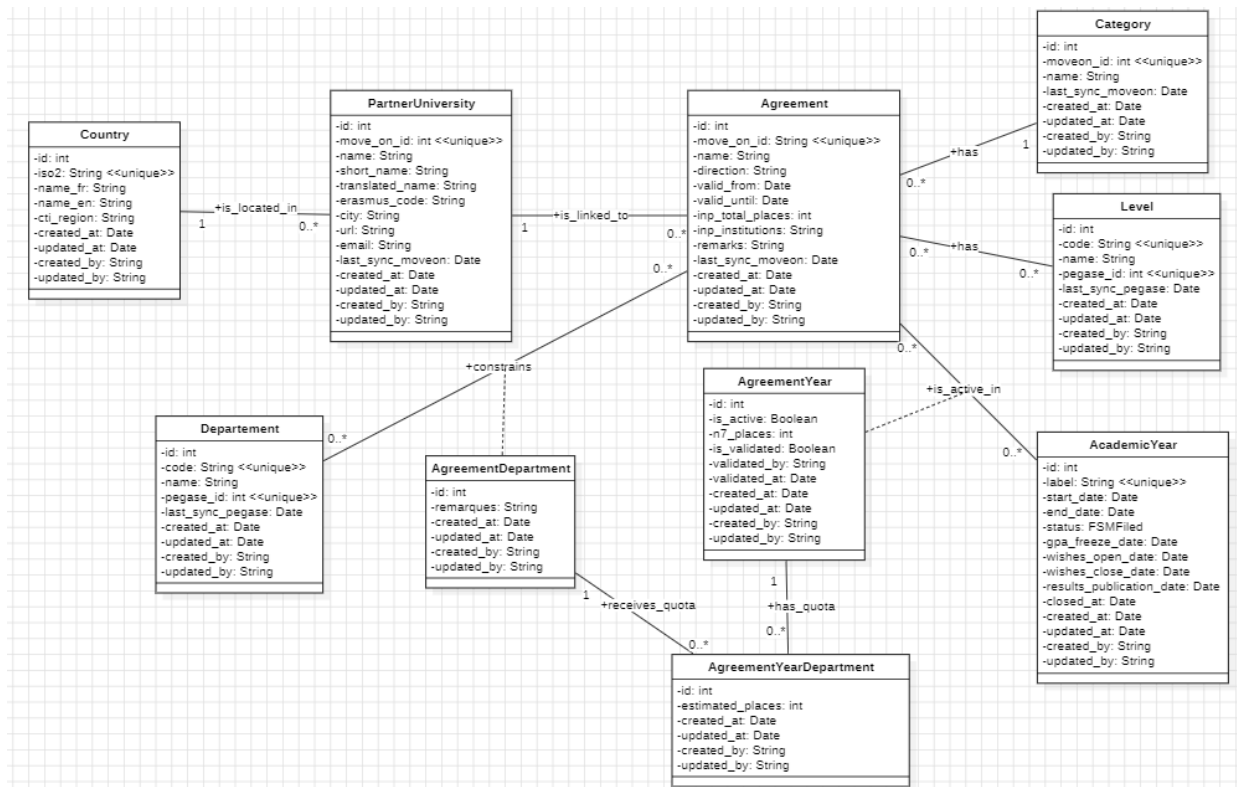


FIGURE 7.1 – Diagramme de classes — Sprint 1

7.1.4 Implémentation

7.1.5 Tests

7.2 Sprint 2 — Données et traçabilité

Cette section présente la réalisation du Sprint 2, consacré à la gestion des étudiants, de leurs inscriptions annuelles et de leurs vœux de mobilité. Elle couvre successivement l’objectif du sprint, le backlog, la conception UML, l’implémentation et les tests.

7.2.1 Objectif du sprint

Le Sprint 2 a pour objectif de rendre le système opérationnel pour la collecte et la centralisation des données. Il couvre quatre user stories : la traçabilité des actions via l’audit log (US-04), l’import et la synchronisation des données académiques étudiants depuis Pégase (US-05), l’import des dossiers de candidature et vœux depuis MoveON (US-06), et la génération des rapports d’import (US-07). Ces quatre user stories sont le prérequis indispensable au Sprint 3 : sans données étudiants fiables issues de Pégase, le gel du GPA et l’algorithme Gale-Shapley ne peuvent pas s’exécuter.

Définition of Done. À l’issue de ce sprint, les imports Pégase et MoveON s’exécutent sans erreur, les données sont persistées en base et rattachées à l’année universitaire cou-

rante, un rapport d'import est généré pour chaque synchronisation, et toute action d'un administrateur RI est tracée dans l'audit log avec l'auteur et l'horodatage.

7.2.2 Sprint Backlog — Sprint 2

#	Tâche	Description	Critères d'acceptation	Prior.	SP
US-04 — Audit log et traçabilité — <i>En tant qu'administrateur RI, je veux que chaque action importante (import, transition FSM, modification, validation) soit enregistrée avec l'auteur et l'horodatage, afin de garantir la traçabilité complète des décisions.</i>					
01	Définir AuditLog	Créer le modèle avec <code>user</code> , <code>action</code> , <code>entity_type</code> , <code>entity_id</code> , <code>payload JSONB</code> et <code>timestamp</code> . Connecter via signal Django <code>post_save</code> sur les modèles critiques.	Le modèle migre sans erreur. Chaque transition FSM et chaque import créent une entrée AuditLog avec l'auteur.	Haute	2
02	Endpoint consultation audit	Exposer <code>GET /api/audit-logs</code> avec filtres par date, auteur et type d'entité via Django Ninja.	Les 20 dernières entrées s'affichent en moins de 200 ms. Les filtres date/auteur/type fonctionnent correctement.	Haute	1
US-05 — Données étudiants — <i>En tant qu'administrateur RI, je veux importer et synchroniser les données académiques des étudiants (INE, GPA, département, niveau) depuis Pégase, afin d'alimenter l'algorithme d'affectation avec des données étudiants fiables et à jour.</i>					
03	Créer adaptateur <code>pegase.py</code>	Implémenter l'extraction des données académiques (INE, GPA, département, niveau) depuis l'API Pégase. Stocker la réponse brute en JSONB dans <code>RawImport</code> .	La réponse brute est stockée avec <code>source="pegase"</code> . Retry exponentiel (3 tentatives) en cas d'échec.	Haute	2
04	Définir <code>Student</code> et <code>AnnualEnrollment</code>	Créer le modèle <code>Student</code> (INE comme clé unique, nom, prénom, email institutionnel) et <code>AnnualEnrollment</code> (inscription annuelle avec GPA, département, niveau, rattaché à <code>AcademicYear</code>).	Les modèles migrent sans erreur. Un étudiant peut avoir plusieurs inscriptions sur des années différentes. INE est unique sur <code>Student</code> .	Haute	2

#	Tâche	Description	Critères d'acceptation	Prior.	SP
05	Pipeline ETL Pégase	Transformer les données brutes en entités <code>Student</code> et <code>AnnualEnrollment</code> . Upsert sur INE. Le GPA est stocké tel quel (gel ultérieur en Sprint 3).	Les étudiants sont créés ou mis à jour. Les doublons sont ignorés. Un rapport d'import est généré avec les anomalies détectées.	Haute	3
06	Interface consultation étudiants	Créer une page Next.js listant les étudiants importés avec leur GPA, département et année d'inscription courante.	La liste s'affiche avec filtre par département et niveau. Les données proviennent de l'API.	Haute	1
US-06 — Vœux étudiants — <i>En tant qu'administrateur RI, je veux importer les dossiers de candidature et les vœux ordonnés des étudiants depuis MoveON, et les consulter par étudiant, afin de les intégrer dans le système d'affectation.</i>					
07	Définir <code>ApplicationFile</code> et <code>Wish</code>	Créer le modèle <code>ApplicationFile</code> (dossier de candidature, FSM à 4 états : <code>open</code> → <code>submitted</code> → <code>processing</code> → <code>closed</code>) et <code>Wish</code> (rang, université cible, département préféré, rattaché à <code>AcademicYear</code>).	Les modèles migrent sans erreur. Les transitions FSM invalides lèvent <code>TransitionNotAllowed</code> . Un <code>Wish</code> ne peut exister sans <code>ApplicationFile</code> .	Haute	2
08	Pipeline ETL vœux MoveON	Importer les vœux depuis MoveON, croiser les INE avec la table <code>Student</code> , rejeter les entrées sans correspondance INE.	Les vœux sont persistés avec rang et université cible. Les INE introuvables sont consignés dans le rapport avec motif <code>INE_NOT_FOUND</code> .	Haute	2
09	Interface consultation vœux	Créer une page Next.js listant les dossiers de candidature par étudiant avec le détail des vœux et l'état FSM de chaque dossier.	La liste s'affiche avec filtre par étudiant et état. Les données sont chargées depuis l'API.	Haute	1
US-07 — Rapport d'import — <i>En tant qu'administrateur RI, je veux consulter un rapport détaillé après chaque import (succès, erreurs, doublons) avec la raison précise de chaque rejet, afin de corriger les anomalies rapidement.</i>					
10	Définir <code>ImportReport</code>	Créer le modèle avec <code>source</code> , <code>total</code> , <code>success_count</code> , <code>error_count</code> , <code>duplicate_count</code> et <code>errors</code> JSONB (motif et identifiant par entrée rejetée), rattaché à <code>RawImport</code> .	Chaque pipeline ETL crée automatiquement un rapport. Les erreurs contiennent motif et numéro de ligne ou identifiant source.	Haute	1

#	Tâche	Description	Critères d'acceptation	Prior.	SP
11	Page rapport d'import	Créer une interface Next.js listant les rapports par date et source avec les compteurs synthétiques et le détail des erreurs accessible par clic.	Les rapports s'affichent triés par date. Le détail des erreurs est accessible. Le filtre par source fonctionne.	Haute	2

7.2.3 Conception détaillée

Cette section présente la conception UML détaillée des entités métier du Sprint 2.

7.2.3.1 Diagramme de classes

Le diagramme de la figure 7.2 modélise la gestion des étudiants et de leurs vœux de mobilité au sein du système de mobilité de l'ENSEEIH.

AnnualEnrollment est la classe d'association centrale du sprint : elle matérialise l'inscription d'un étudiant (**Student**) pour une année universitaire (**AcademicYear**) donnée, et porte le département et le niveau d'études. Le parcours est renseigné uniquement lorsque l'étudiant a intégré une filière spécialisée ; il reste absent pour les niveaux en tronc commun (première année). **StudentWish** matérialise à son tour un vœu de mobilité émis dans le cadre d'une inscription annuelle : chaque inscription peut générer au plus trois vœux, chacun pointant vers un accord différent et portant un rang et un statut.

Une contrainte d'intégrité garantit que le parcours suivi par l'étudiant appartient au même département que son inscription annuelle, exprimée par la contrainte :

`{parcours.department = department}` annotée sur l'association.

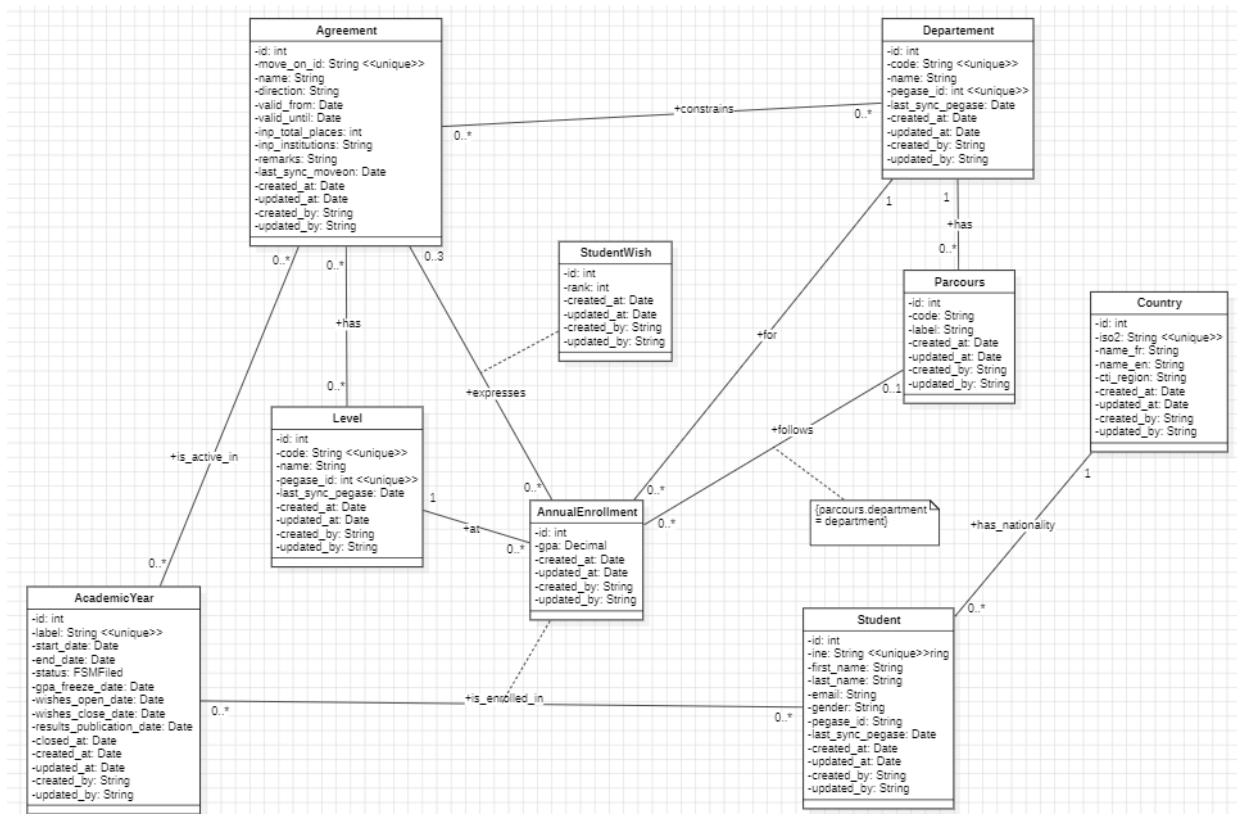


FIGURE 7.2 – Diagramme de classes — Sprint 2

7.2.4 Implémentation

7.2.5 Tests

7.3 Sprint 3 — Algorithme d'affectation

Cette section présente la réalisation du Sprint 3, consacré à l'algorithme d'affectation et à la publication des résultats. Elle couvre successivement l'objectif du sprint, le backlog, la conception UML, l'implémentation et les tests.

7.3.1 Objectif du sprint

Le Sprint 3 a pour objectif de produire et de publier les propositions d'affectation. Il couvre trois user stories, réalisées dans l'ordre suivant : l'implémentation de l'algorithme Deferred Acceptance (Gale-Shapley) (US-08), la supervision manuelle des propositions par le service RI avant publication (US-09), et la consultation des résultats par les étudiants une fois les affectations publiées (US-10).

Ce sprint constitue le cœur fonctionnel du système : toutes les données collectées dans les sprints précédents — GPA importés depuis Pégase et vœux synchronisés depuis MoveON — convergent ici pour produire un résultat actionnable.

Définition of Done. À l’issue de ce sprint, l’algorithme Gale-Shapley doit s’exécuter sans erreur et produire des affectations stables respectant les quotas par accord et par département, les propositions doivent être consultables et modifiables par le service RI avant publication, et chaque étudiant doit pouvoir consulter son affectation après publication des résultats.

7.3.2 Sprint Backlog — Sprint 3

#	Tâche	Description	Critères d’acceptation	Prior.	SP
US-08 — Algorithme Gale-Shapley — <i>En tant qu’administrateur RI, je veux lancer l’algorithme d’affectation (Deferred Acceptance) qui affecte chaque étudiant à son meilleur vœu disponible en respectant les quotas par département, afin de produire une proposition équitable et stable.</i>					
01	Créer le modèle Assignment	Définir Assignment avec student, agreement, rank, status (FSM : proposed → validated → published cancelled) et override_reason. Générer et appliquer la migration.	Le modèle migre sans erreur. Contrainte unique sur (student, academic_year). Les transitions FSM invalides lèvent TransitionNotAllowed.	Haute	2
02	Implémenter l’algorithme Deferred Acceptance	Coder gale_shapley() en Python pur. L’algorithme parcourt les vœux par rang et respecte les deux niveaux de quotas : quota N7 par accord et quota par département (issus de US-03 pour ce sprint). Les non-affectés reçoivent un motif (NO_QUOTA, NO_WISH). En post-traitement, les places non utilisées par les départements sous-demandés sont redistribuées.	Les affectations sont stables (aucune paire bloquante). Les quotas ne sont jamais dépassés. Exécution < 30 s pour 500 étudiants. La redistribution du surplus est tracée dans l’audit log.	Haute	5
03	Encapsuler dans une tâche Django-Q2	Créer la tâche asynchrone déclenchée par la transition FSM launch_pre_assignment. Retourner un identifiant de tâche (HTTP 202) immédiatement.	La transition FSM déclenche la tâche. Le résultat est lisible après complétion sans relancer l’algorithme.	Haute	2

#	Tâche	Description	Critères d'acceptation	Prior.	SP
04	Exposer les endpoints REST des affectations	Créer GET <code>/api/assignments/</code> , GET <code>/api/assignments/{id}/</code> et GET <code>/api/assignments/stats/</code> avec filtres par département et statut.	Réponses paginées. Non-affectés retournés avec motif. Statistiques cohérentes avec la liste.	Haute	2
05	Page résultats d'affectation (admin RI)	Créer une page Next.js affichant le tableau des affectations, le taux de remplissage des quotas par accord et la liste des non-affectés avec motif.	Le tableau se charge depuis l'API. Le statut d'exécution (en cours / terminé / non lancé) est affiché.	Haute	2
US-09 — Supervision des affectations — <i>En tant qu'administrateur RI, je veux consulter les propositions d'affectation, les ajuster manuellement pour les cas particuliers et tracer chaque modification avec un motif, avant de valider et publier les résultats.</i>					
06	Page de supervision des affectations	Lister les propositions par département avec le rang du vœu retenu, l'université cible et le statut. Filtres par département, statut et nom d'étudiant.	Liste paginée chargée depuis l'API. Statuts proposed , validated et published visuellement distincts.	Haute	3
07	Modifier manuellement une affectation	Permettre à l'admin RI de changer l'université cible ou d'annuler une affectation. Motif (<code>override_reason</code>) obligatoire, tracé dans l'AuditLog.	Modification refusée sans motif. Entrée d'audit créée avec valeurs avant/après. Impossible si published .	Haute	2
08	Valider et publier les affectations	Exposer POST <code>/api/assignments/validate-all/</code> et POST <code>/api/assignments/publish/</code> . Déclencher <code>submit_for_validation</code> sur la FSM de l'année universitaire. Ajouter les boutons dans l'interface.	Publication déclenche la transition FSM. Affectations publiées en lecture seule. Bouton grisé si affectations incomplètes.	Haute	3
US-10 — Consultation des résultats — <i>En tant qu'étudiant, je veux consulter mon affectation une fois les résultats publiés afin de connaître ma destination et les démarches à suivre.</i>					
09	Endpoint résultat de l'étudiant authentifié	Exposer GET <code>/api/students/me/assignment/</code> avec contrôle d'accès par objet : un étudiant ne consulte que sa propre affectation.	404 si résultats non publiés. 403 si accès à l'affectation d'un autre étudiant.	Haute	1

#	Tâche	Description	Critères d'acceptation	Prior.	SP
10	Page consultation résultats (espace étudiant)	Créer une page Next.js affichant l'université cible, le département, la période et les démarches à suivre. Message de recours affiché si l'étudiant n'est pas affecté.	Page accessible uniquement après publication. Motif de rejet visible pour les non-affectés.	Haute	2

7.3.3 Conception détaillée

7.3.3.1 Diagramme de classes

7.3.4 Implémentation

7.3.5 Tests

Conclusion générale

Bibliographie

- [1] *L'ENSEEIH*T – *Présentation*. <https://www.enseeiht.fr/fr/l-enseeiht.html>. Consulté le 2026-03-18.
- [2] *International* – *Toulouse INP-ENSEEIH*T. <https://www.enseeiht.fr/fr/international.html>. Consulté le 2026-03-18.
- [3] *Mobilité sortante* – *Toulouse INP-ENSEEIH*T. <https://www.enseeiht.fr/fr/international/mobilite-sortante.html>. Consulté le 2026-03-18.
- [4] *Mobilité entrante* – *Toulouse INP-ENSEEIH*T. <https://www.enseeiht.fr/fr/international/mobilite-entrante.html>. Consulté le 2026-03-18.
- [5] *Programmes courts* – *Toulouse INP-ENSEEIH*T. <https://www.enseeiht.fr/fr/international/programmes-courts.html>. Consulté le 2026-03-18.
- [6] *MoveON* – *QS Solutions*. <https://www.qs.com/solutions/moveon>. Consulté le 2026-03-09.
- [7] *Définition CRM* – *Eudonet*. <https://www.eudonet.com/fr/ressources/blog/definition-crm/>. Consulté le 2026-03-09.
- [8] Mehdi ELAHI et al. « Developing and Evaluating a University Recommender System ». In : *Frontiers in Artificial Intelligence* (2022). Consulté le 2026-03-09. URL : <https://pmc.ncbi.nlm.nih.gov/articles/PMC8848746/>.
- [9] *React* – *The library for web and native user interfaces*. <https://react.dev/>. Consulté le 2026-03-17.
- [10] *Angular* – *The web development framework*. <https://angular.dev/>. Consulté le 2026-03-17.
- [11] *Next.js* – *The React Framework for the Web*. <https://nextjs.org/>. Consulté le 2026-03-17.
- [12] *Node.js* – *Run JavaScript Everywhere*. <https://nodejs.org/>. Consulté le 2026-03-18.
- [13] *FastAPI* – *Modern, fast web framework for building APIs with Python*. <https://fastapi.tiangolo.com/>. Consulté le 2026-03-18.
- [14] *Django* – *The web framework for perfectionists with deadlines*. <https://www.djangoproject.com/>. Consulté le 2026-03-18.
- [15] *Django REST Framework* – *Web APIs for Django*. <https://www.django-rest-framework.org/>. Consulté le 2026-03-18.
- [16] *Django Ninja* – *Fast Django REST Framework*. <https://django-ninja.dev/>. Consulté le 2026-03-18.

- [17] *MySQL – The world’s most popular open source database.* <https://www.mysql.com/>. Consulté le 2026-03-17.
- [18] *MongoDB – The developer data platform.* <https://www.mongodb.com/>. Consulté le 2026-03-17.
- [19] *PostgreSQL – The world’s most advanced open source relational database.* <https://www.postgresql.org/>. Consulté le 2026-03-17.
- [20] *Celery – Distributed Task Queue.* <https://docs.celeryq.dev/>. Consulté le 2026-03-19.
- [21] *Redis – The Real-time Data Platform.* <https://redis.io/docs/>. Consulté le 2026-03-19.
- [22] *Django Q2 – A multiprocessing task queue for Django.* <https://django-q2.readthedocs.io/>. Consulté le 2026-03-19.
- [23] *MinIO – High Performance Object Storage.* <https://min.io/docs/>. Consulté le 2026-03-24.
- [24] *Nginx – High Performance Load Balancer, Web Server, & Reverse Proxy.* <https://nginx.org/en/docs/>. Consulté le 2026-03-27.
- [25] *Traefik – The Cloud Native Application Proxy.* <https://doc.traefik.io/traefik/>. Consulté le 2026-03-27.
- [26] *Scikit-learn – Machine Learning in Python.* <https://scikit-learn.org/stable/>. Consulté le 2026-03-24.
- [27] *Mistral AI – La plateforme pour les modèles de langage.* <https://docs.mistral.ai/>. Consulté le 2026-03-30.
- [28] *LangChain – Build context-aware reasoning applications.* <https://python.langchain.com/docs/>. Consulté le 2026-03-30.
- [29] *LlamaIndex – Python Framework Documentation.* <https://developers.llamaindex.ai/python/framework/>. Consulté le 2026-03-30.
- [30] *Haystack – The open-source AI framework.* <https://docs.haystack.deepset.ai/>. Consulté le 2026-03-30.